Exploratory-Analysis-of-NYC-Taxi / NYC Taxi Trip Duration.ipynb

TanmayKedari Add files via upload fc3a458 · 5 years ago

4161 lines (4161 loc) · 1.24 MB

NYC Taxi Trip Duration Prediction

Contents

Problem Statement

1. Exploratory Data Analysis on NYC Taxi data
2. Build a machine learning model to predict the duration of NYC taxi trip.

Data Exploration

Libraries

In [1]:
```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import datetime as dt
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.ensemble import AdaBoostRegressor
```

```
from sklearn.ensemble import GradientBoostingRegressor
from xgboost import XGBRegressor
from sklearn import metrics
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
from geopy.distance import geodesic
import time
from IPython.display import clear_output          #For Jupyter Notebook
```

Import Data

Import the data from a csv file.

In [2]:
```
df = pd.read_csv('/home/tanx/DBDA/CDAC_Project/Final/train.csv')
```

Explore Data

In [19]:
```
df.dtypes
```

Out[19]:
```
id                   object
vendor_id             int64
pickup_datetime      object
dropoff_datetime     object
passenger_count       int64
pickup_longitude    float64
pickup_latitude     float64
dropoff_longitude   float64
dropoff_latitude    float64
store_and_fwd_flag   object
trip_duration         int64
dtype: object
```

In [20]:
```
df.head()
```

Out[20]:

	id	vendor_id	pickup_datetime	dropoff_datetime	passenger_count	pickup
0	id2875421	2	2016-03-14 17:24:55	2016-03-14 17:32:30	1	
1	id2377394	1	2016-06-12 00:43:35	2016-06-12 00:54:38	1	
2	id3858529	2	2016-01-19 11:35:24	2016-01-19 12:10:48	1	
3	id3504673	2	2016-04-06 19:32:31	2016-04-06 19:39:40	1	
4	id2181028	2	2016-03-26 13:30:55	2016-03-26 13:38:10	1	

In [21]:
```
df.tail()
```

Out[21]:

	id	vendor_id	pickup_datetime	dropoff_datetime	passenger_count
1458639	id2376096	2	2016-04-08 13:31:04	2016-04-08 13:44:02	4
1458640	id1049543	1	2016-01-10 07:35:15	2016-01-10 07:46:10	1
1458641	id2304944	2	2016-04-22 06:57:41	2016-04-22 07:10:25	1
1458642	id2714485	1	2016-01-05 15:56:26	2016-01-05 16:02:39	1
1458643	id1209952	1	2016-04-05 14:44:25	2016-04-05 14:47:43	1

1.First and most important thing checking for null values

In [22]:
```
df.isna().sum()
```

Out[22]:
```
id                  0
vendor_id           0
pickup_datetime     0
dropoff_datetime    0
passenger_count     0
pickup_longitude    0
pickup_latitude     0
dropoff_longitude   0
dropoff_latitude    0
store_and_fwd_flag  0
trip_duration       0
dtype: int64
```

There are no Null Values

2.Convert timestamp to datetime format to fetch the other details as listed below

In [141…
```
df['pickup_datetime'] = pd.to_datetime(df['pickup_datetime'])
df['dropoff_datetime'] = pd.to_datetime(df['dropoff_datetime'])
```

In [142…
```
#Calculate and assign new columns to the dataframe such as weekday,
#month and pickup_hour which will help us to gain more insights from the data
df['weekday'] = df.pickup_datetime.dt.weekday_name
df['month'] = df.pickup_datetime.dt.month
df['weekday_num'] = df.pickup_datetime.dt.weekday
df['pickup_hour'] = df.pickup_datetime.dt.hour
```

3.Calculate distance between pickup and dropoff coordinates using geodesic.

In [ ]:
```
distance = []
for index in df['pickup_latitude'].index:
    clear_output()
    print(index)
    distance.append(geodesic((df['pickup_latitude'].iloc[index],df['pickup_lo
df['distance'] = distance
```

4.Calculate Speed in miles/hr for further insights

In [47]:
```
df['speed'] = (df.distance/(df.trip_duration/3600))
```

5.Add 2 more column of PickUp Location Id and DropOff Location Id based on Zone Location Id

In [10]:
```
loc = pd.read_csv('/home/tanx/Documents/taxi_zones.csv')
```

In [ ]:
```
list1 = []
for index in df.iloc[:,0].index:
    clear_output()
    print(index)
    for ind in loc.iloc[:,0].index:
        if df['pickup_longitude'].iloc[index].round(2) == loc['Longitude'].il
            list1.append(loc['LocationID'].iloc[ind])
            break
```

In [ ]:
```
df['PULocationID'] = list1
```

In [ ]:
```
list2 = []
for index in df.iloc[:,0].index:
    clear_output()
    print(index)
    for ind in loc.iloc[:,0].index:
        if df['dropoff_longitude'].iloc[index].round(2) == loc['Longitude'].i
            list2.append(loc['LocationID'].iloc[ind])
            break
```

In [ ]:
```
df['DOLocationID'] = list2
```

Column Details

- id - a unique identifier for each trip
- vendor_id - a code indicating the provider associated with the trip record
- pickup_datetime - date and time when the meter was engaged
- dropoff_datetime - date and time when the meter was disengaged
- passenger_count - the number of passengers in the vehicle (driver entered value)
- pickup_longitude - the longitude where the meter was engaged
- pickup_latitude - the latitude where the meter was engaged
- dropoff_longitude - the longitude where the meter was disengaged
- dropoff_latitude - the latitude where the meter was disengaged
- store_and_fwd_flag - This flag indicates whether the trip record was held in vehicle memory before sending to the vendor because the vehicle did not have a connection to the server - Y=store and forward; N=not a store and forward trip.
- trip_duration - duration of the trip in seconds

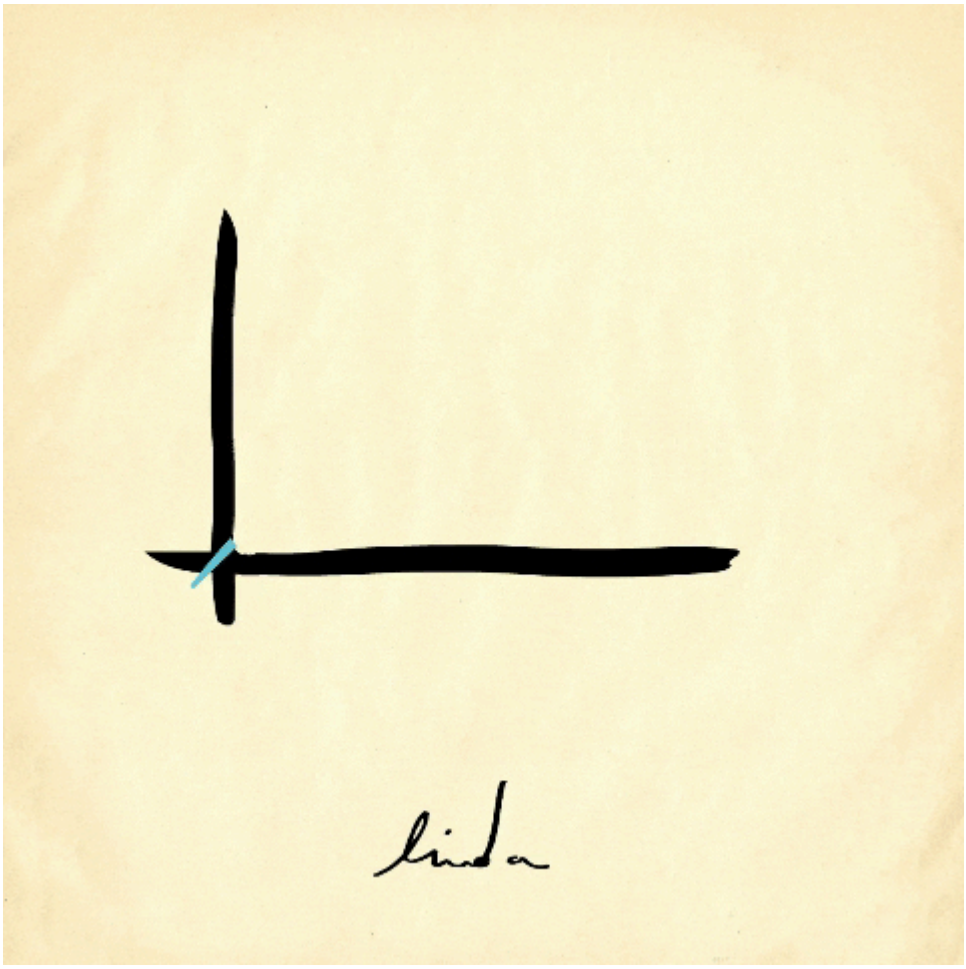
- trip_duration - duration of the trip in seconds
- distance - geographic distance between two co-ordinates
- speed - average spped of the trip (in miles/hr)

Data Analysis

This section represent the Univariate and Bivariate analysis of the data

Univariate Analysis

Univariate analysis is the analysis of one variable. It's major purpose is to describe patterns in the data consisting of single variable.



1. Id

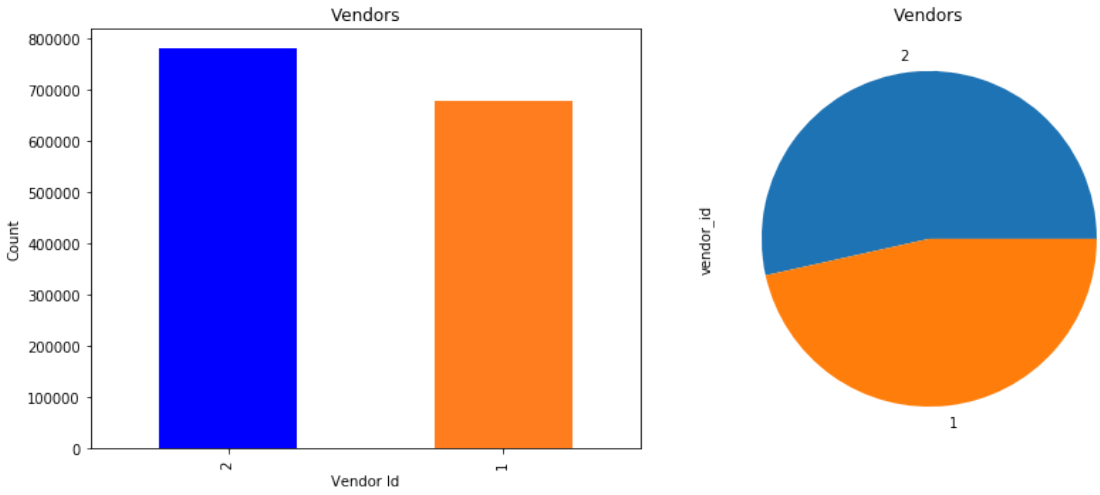
There are 1393253 Unique id's which represent each row in the data

2. Vender Id

A code indicating the provider associated with the trip record

In [4]:

```
fig, axes = plt.subplots(nrows=1, ncols=2,figsize=(12,5))
ax = df['vendor_id'].value_counts().plot(kind='bar',title="Vendors",ax=axes[0
df['vendor_id'].value_counts().plot(kind='pie',title="Vendors",ax=axes[1])
ax.set_ylabel("Count")
ax.set_xlabel("Vendor Id")
fig.tight_layout()
```



- Here we got to know that there are only 2 venders(1 and 2)
- Both the venders share almost equal amount of trips, the difference is quite low between two venders
- But Vendor 2 is evidently more famous among the population as per the above graphs.

3.Passengers

New York City Taxi Passenger Limit says:

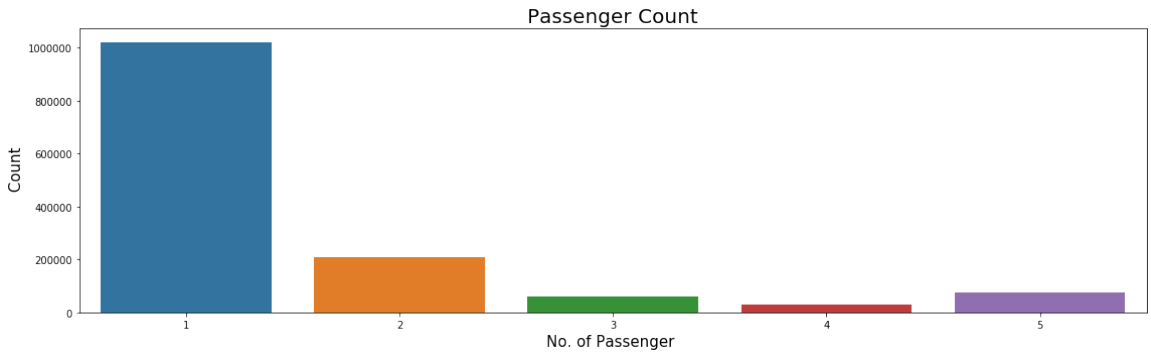
- A maximum of 4 passengers can ride in traditional cabs.
- A child under 7 is allowed to sit on a passenger's lap in the rear seat in addition to the passenger limit.

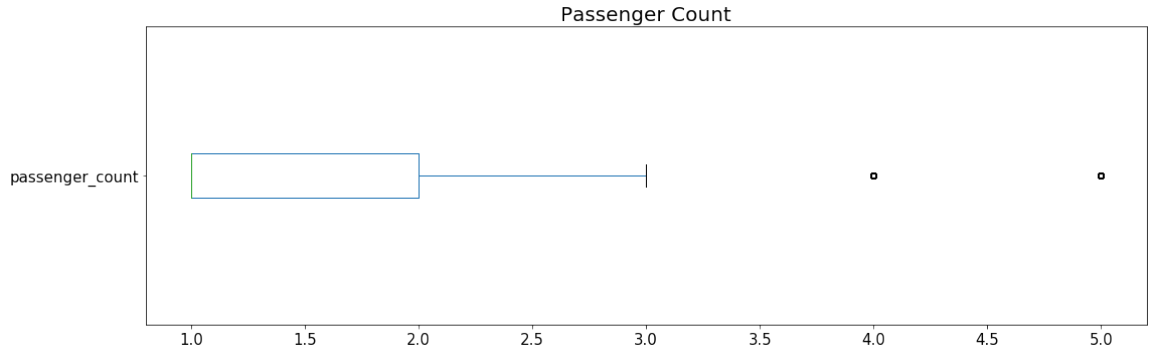
So, in total we can assume that maximum 5 passenger can board the new york taxi i.e. 4 adult + 1 minor

In [5]:
```
pd.options.display.float_format = '{:.2f}'.format #To suppress scientific not
df.passenger_count.value_counts()
```

Out[5]:
```
1    1033540
2     210318
5      78088
3      59896
6      48333
4      28404
0         60
7          3
9          1
8          1
Name: passenger_count, dtype: int64
```

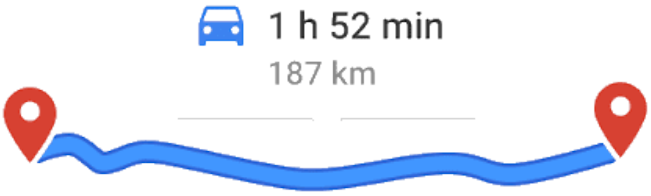
In [3]:
```
fig, axes = plt.subplots(nrows=1, ncols=1,figsize=(16,5))
# line = df['passenger_count'].value_counts().plot(kind='bar',fontsize = 15)
line = sns.countplot(df.passenger_count)
line.set_ylabel(" Count",fontsize = 15)
line.set_xlabel("No. of Passenger ",fontsize = 15)
line.set_title('Passenger Count',fontsize = 20)
fig.tight_layout()
fig, axes = plt.subplots(nrows=1, ncols=1,figsize=(16,5))
box = df['passenger_count'].plot(kind='box',vert = False,fontsize = 15)
box.set_title('Passenger Count',fontsize = 20)
fig.tight_layout()
```



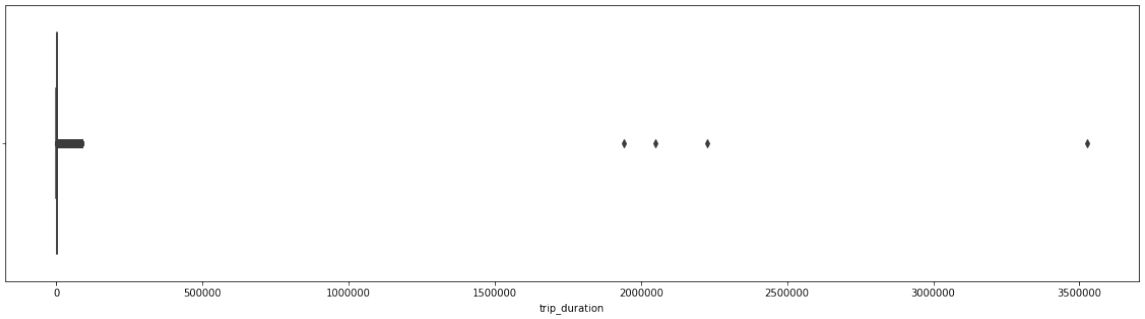


- There are some trips with 0 passenger count.
- Few trips consisted of even 6, 7, 8 or 9 passengers. Clear outliers and pointers to data inconsistency
- Most of trip consist of passenger either 1 or 2.

4. Trip duration



In [91]:
```
plt.figure(figsize = (20,5))
sns.boxplot(df.trip_duration)
plt.show()
```



- Some trip durations are over 100000 seconds which are clear outliers and should be removed.
- There are some durations with as low as 1 second. which points towards trips with 0 km distance.
- Major trip durations took between 10-20 mins to complete.
- Mean and mode are not same which shows that trip duration distribution is skewed towards right

Let's analyze more

In [93]:
```
df.trip_duration.groupby(pd.cut(df.trip_duration, np.arange(1,max(df.trip_dur
```

Out[93]:
```
trip_duration
(1, 3601]             1446313
(3601, 7201]            10045
(7201, 10801]             141
(10801, 14401]             35
(14401, 18001]              5
                         ...
```

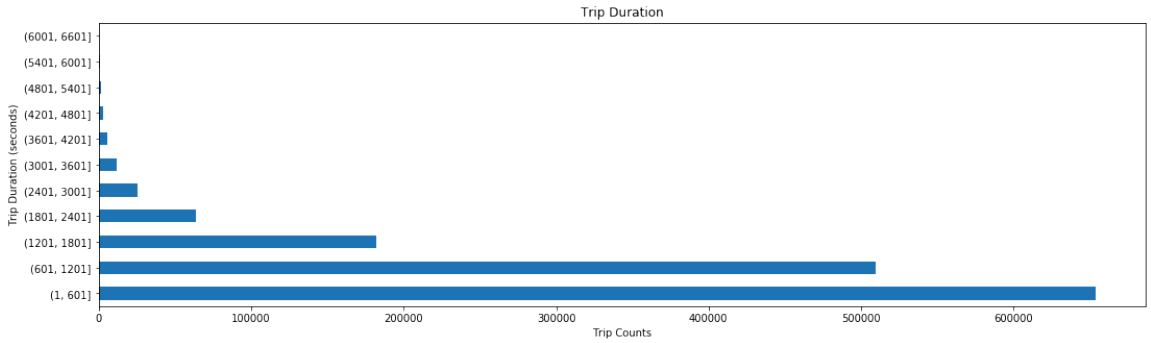
```
(3506401, 3510001]          0
(3510001, 3513601]          0
(3513601, 3517201]          0
(3517201, 3520801]          0
(3520801, 3524401]          0
Name: trip_duration, Length: 979, dtype: int64
```

- These trips ran for more than 20 days, which seems unlikely by the distance travelled.
- All the trips are taken by vendor 1 which points us to the fact that this vendor might allows much longer trip for outstations.
- All these trips are either taken on Tuesday's in 1st month or Saturday's in 2nd month. There might be some relation with the weekday, pickup location, month and the passenger.
- But they fail our purpose of correct prediction and bring inconsistencies in the algorithm calculation.

Let's visualize the number of trips taken in slabs of 0-10, 20-30 ... minutes respectively

In [110…

```
df.trip_duration.groupby(pd.cut(df.trip_duration, np.arange(1,7200,600))).cou
plt.title('Trip Duration')
plt.xlabel('Trip Counts')
plt.ylabel('Trip Duration (seconds)')
plt.show()
```



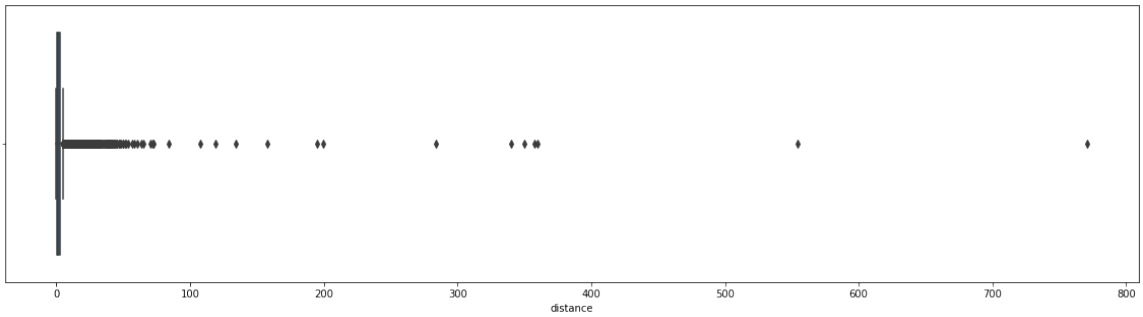
We can observe that most of the trips took 0 - 30 mins to complete i.e. approx 1800 secs.

5.Distance

Let's now have a look on the distribution of the distance across the different types of rides.

In [115…

```
plt.figure(figsize = (20,5))
sns.boxplot(df.distance)
plt.show()
```



Interesting find:

- There some trips with over 60 miles distance.
- Some of the trips distance value is 0 miles.

Observations:

- mean distance travelled is approx 2.1 miles.

In [139…

```
print(f"There are {df.distance[df.distance == 0 ].count()} trip records with
```

```
There are 5897 trip records with 0 miles distance
```

- Around 6K trip record with distance equal to 0. Below are some possible explanation for such records.

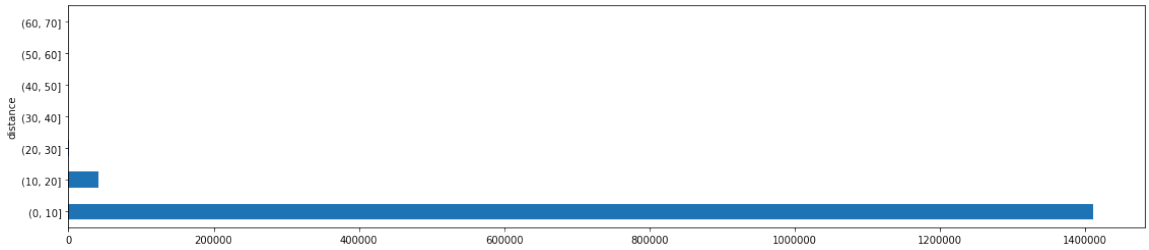
1. Customer changed mind and cancelled the journey just after accepting it.
2. Software didn't recorded dropoff location properly due to which dropoff location is the same as the pickup location.
3. Issue with GPS tracker while the journey is being finished.
4. Driver cancelled the trip just after accepting it due to some reason. So the trip couldn't start
5. Or some other issue with the software itself which a technical guy can explain

There is some serious inconsistencies in the data where drop off location is same as the

pickup location. We can't think off imputing the distance values considering a correlation with the duration because the dropoff_location coordinates would not be inline with the distance otherwise. We will look more to it in bivariate analysis with the Trip duration.

In [126…

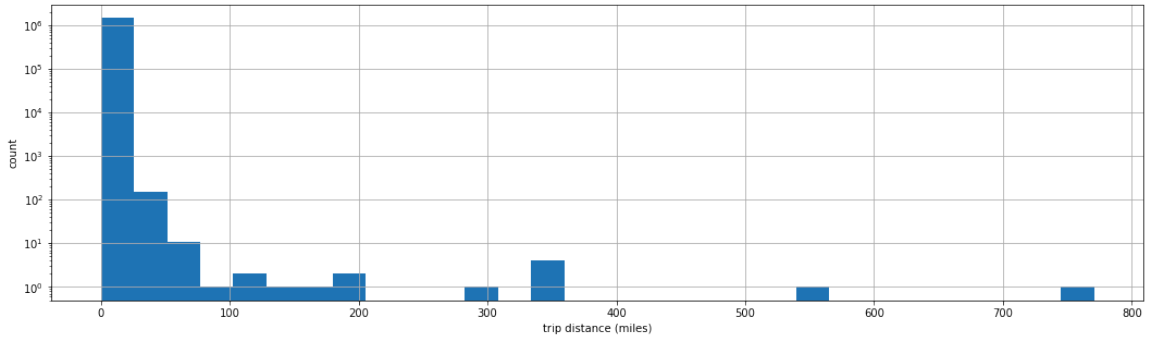
```
df.distance.groupby(pd.cut(df.distance, np.arange(0,80,10))).count().plot(kin
plt.show()
```



From the above observation it is evident that most of the rides are completed between 1-10 miles with some of the rides with distances between 10-30 miles. Other slabs bar are not visible because the number of trips are very less as compared to these slabs

In [114…

```
ax = df['distance'].hist(bins=30, figsize=(18,5))
ax.set_yscale('log')
ax.set_xlabel("trip distance (miles)")
ax.set_ylabel("count")
plt.show()
```



According to the dustribution of trip distances and the fact that it takes about 30 miles to drive across the whole New York City, we decided to use 30 as the number to split the trips into short or long distance trips.

In [131…

```
df_short = df[df.distance <= 30].count()
df_long = df[df.distance > 30].count()
print(f"Short Trips: {df_short[0]} records in total.\nLong Trips: {df_long[0]
```

```
Short Trips: 1458545 records in total.
Long Trips: 99 records in total.
```

6.Speed

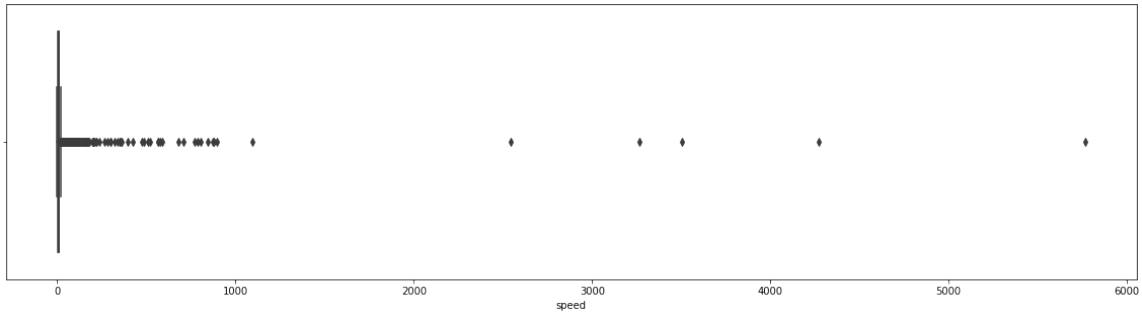
Speed is a function of distance and time. Let's visualize speed in different trips.

Maximum speed limit in NYC is as follows:

- 25 mph in urban area
- 65 mph on controlled state highways

In [134…

```
plt.figure(figsize = (20,5))
sns.boxplot(df.speed)
plt.show()
```

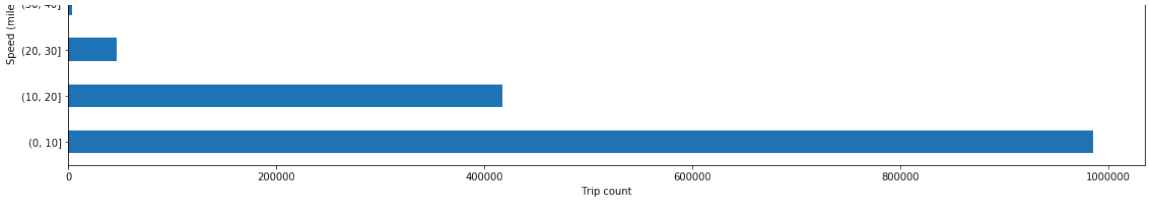


- Many trips were done at a speed of over 125 mile/h. Going SuperSonic..!!

Let's remove them and focus on the trips which were done at less than 65 mile/h as per the speed limits

In [137…

```
df = df[df.speed <= 65]
df.speed.groupby(pd.cut(df.speed, np.arange(0,65,10))).count().plot(kind = 'b
plt.xlabel('Trip count')
plt.ylabel('Speed (mile/H)')
plt.title('Speed')
plt.show()
```



- Trips over 15 miles/h are being considered as outliers but we cannot ignore them because they are well under the highest speed limit of 65 mile/h on state controlled highways.
- Mostly trips are done at a speed range of 6-12 miles/h with an average speed of around 8 miles/h.

It is evident from this graph what we thought off i.e. most of the trips were done at a speed range of 6-12 miles/H.

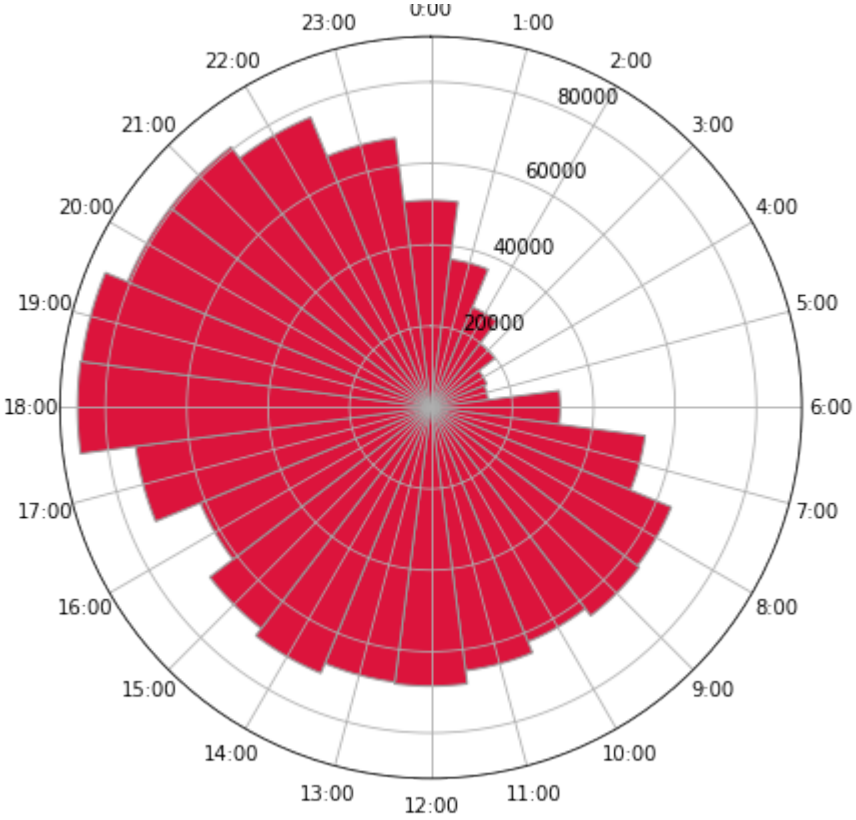
7.Total trips Per Hour

Let's take a look at the distribution of the pickups across the 24 hour time scale.

In [54]:
```
def clock(ax, radii, title, color):
    N = 24
    bottom = 2

    # create theta for 24 hours
    theta = np.linspace(0.0, 2 * np.pi, N, endpoint=False)


    # width of each bin on the plot
    width = (2*np.pi) / N

    bars = ax.bar(theta, radii, width=width, bottom=bottom, color=color, edge

    # set the lable go clockwise and start from the top
    ax.set_theta_zero_location("N")
    # clockwise
    ax.set_theta_direction(-1)

    # set the label
    ax.set_xticks(theta)
    ticks = ["{}:00".format(x) for x in range(24)]
    ax.set_xticklabels(ticks)
    ax.set_title(title)
```

In [55]:
```
plt.figure(figsize = (15,15))
ax = plt.subplot(2,2,1, polar=True)
    # make the histogram that bined on 24 hour
radii = np.array(df['pickup_hour'].value_counts(sort = False).tolist(), dtype
# radii = np.array(df['pickup_hour'].value_counts().tolist(), dtype="int64")
title = "Trips per Hour"
clock(ax, radii, title, "#dc143c")
```

Trips per Hour

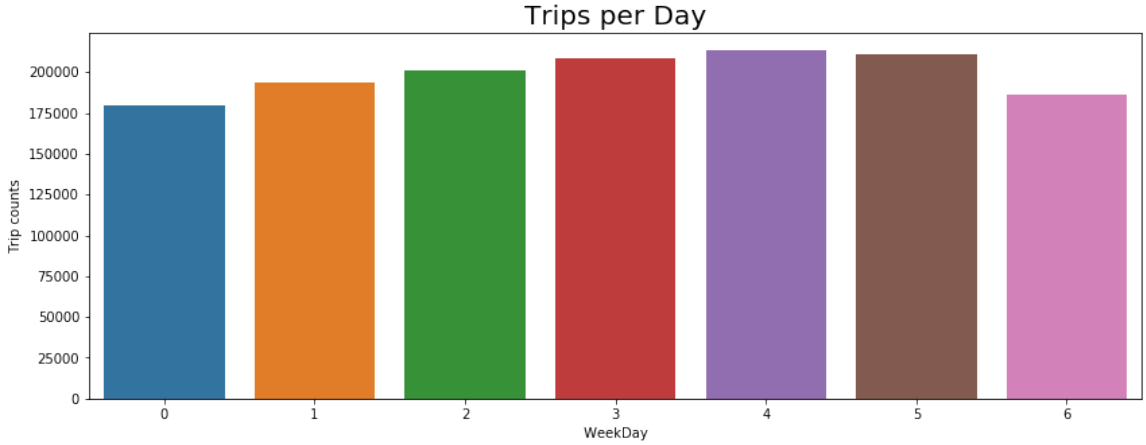


- It's inline with the general trend of taxi pickups which starts increasing from 6AM in the morning and then declines from late evening i.e. around 8 PM. There is no unusual behavior here.
- The number of pickup is maximum at 6-7 pm.

8.Total trips per weekday

Let's take a look now at the distribution of taxi pickups across the week.

In [7]:
```
plt.figure(figsize = (14,5))
sns.countplot(df.weekday_num)
plt.xlabel(' WeekDay ')
plt.ylabel('Trip counts')
plt.title('Trips per Day',fontsize = 20)
plt.show()
```

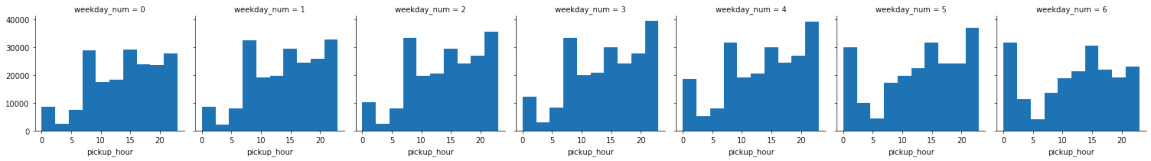


- Here we can see an increasing trend of taxi pickups starting from Monday till Friday. The trend starts declining from saturday till monday which is normal where some office going people likes to stay at home for rest on the weekends.

Let's drill down more to see the hourwise pickup pattern across the week

In [185…

```
n = sns.FacetGrid(df, col='weekday_num')
n.map(plt.hist, 'pickup_hour')
plt.show()
```



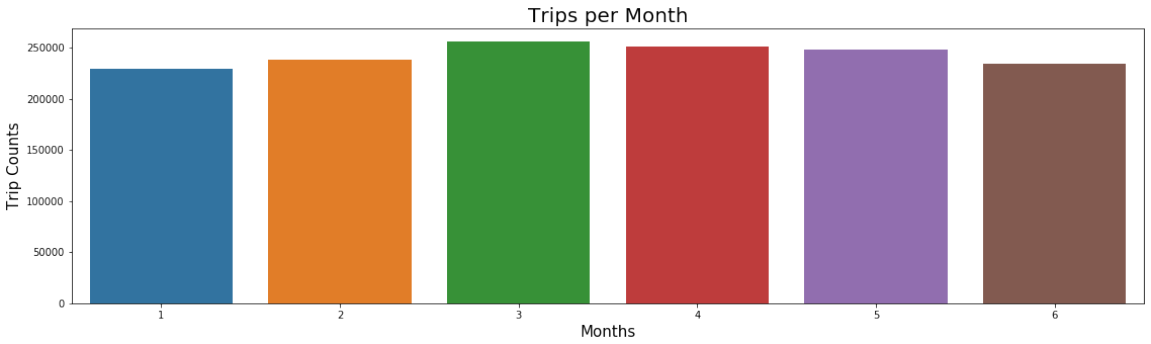
- Taxi pickups increased in the late night hours over the weekend possibly due to more outstation rides or for the late night leisures nearby activities.
- Early morning pickups i.e before 5 AM have increased over the weekend in comparison to the office hours pickups i.e. after 7 AM which have decreased due to obvious reasons.
- Taxi pickups seems to be consistent across the week at 15 Hours i.e. at 3 PM.

9.Total trips per month

Let's take a look at the trip distribution across the months to understand if there is any diffrence in the taxi pickups in different months

In [189…

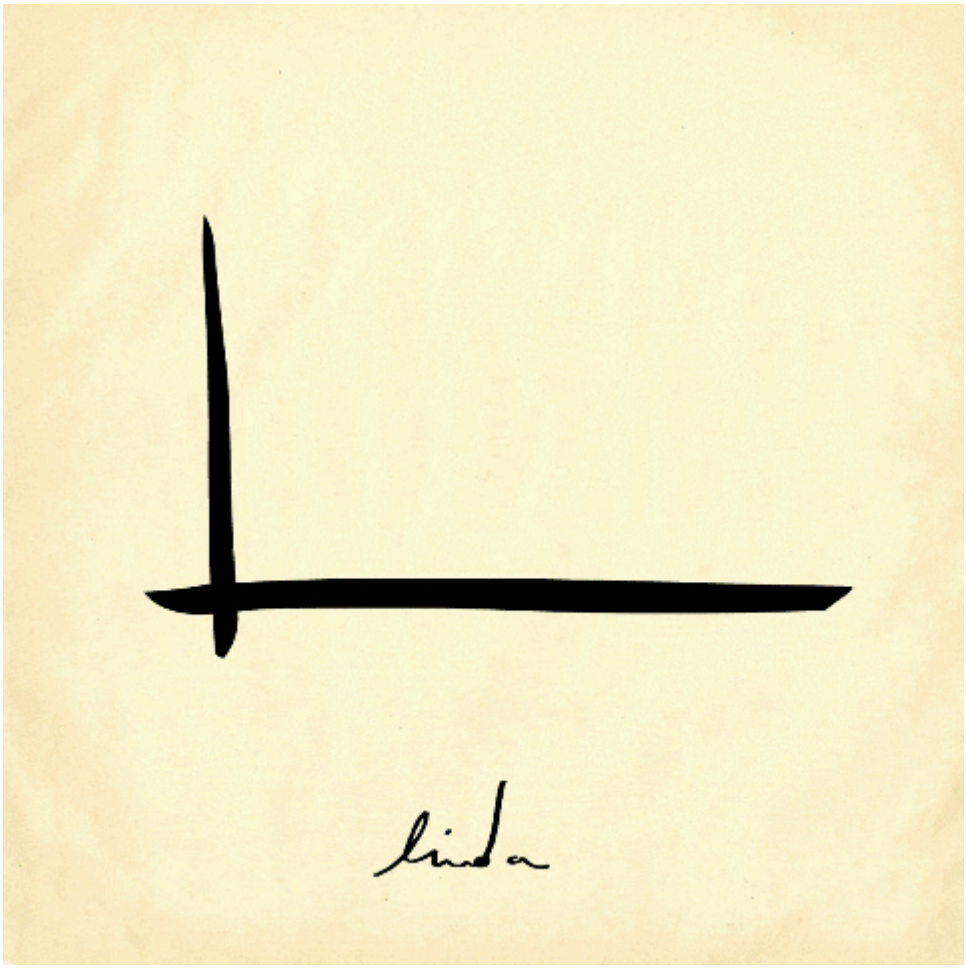
```
plt.figure(figsize = (19,5))
sns.countplot(df.month)
plt.ylabel('Trip Counts',fontsize = 15)
plt.xlabel('Months',fontsize = 15)
plt.title('Trips per Month',fontsize = 20)
plt.show()
```



Quite a balance across the months here. It could have been more equivalent if we wouldn't have removed the inconsistent records in our study of the univariate analysis

Bivariate Analysis

Bivariate analysis is used to find out if there is a relationship between two sets of values. It usually involves the variables X and Y.

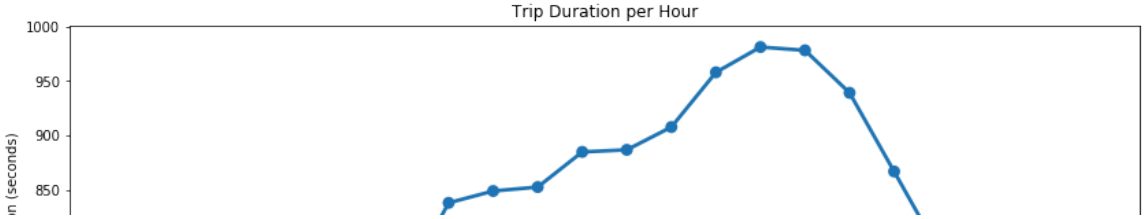


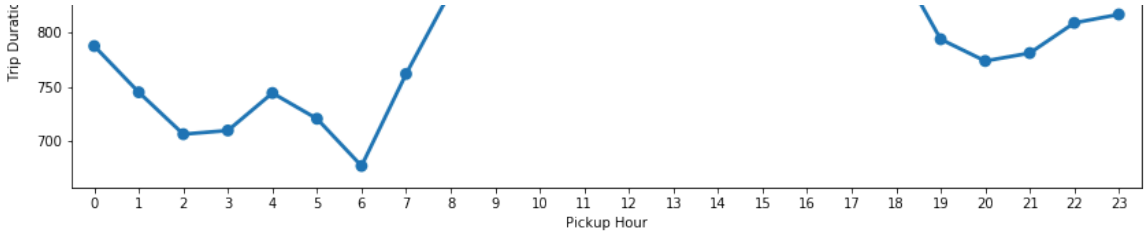
1.Trip Duration per hour

We need to aggregate the total trip duration to plot it agaist the month. The aggregation measure can be anything like sum, mean, median or mode for the duration. Since we already did the outlier analysis, so we can take the mean to visualize the pattern which should not result in the bias of the general trend.

- Lets take a look.

In [9]:
```
plt.figure(figsize = (14,5))
group1 = df.groupby('pickup_hour').trip_duration.mean()
sns.pointplot(group1.index, group1.values)
plt.ylabel('Trip Duration (seconds)')
plt.xlabel('Pickup Hour')
plt.title('Trip Duration per Hour')
plt.show()
```



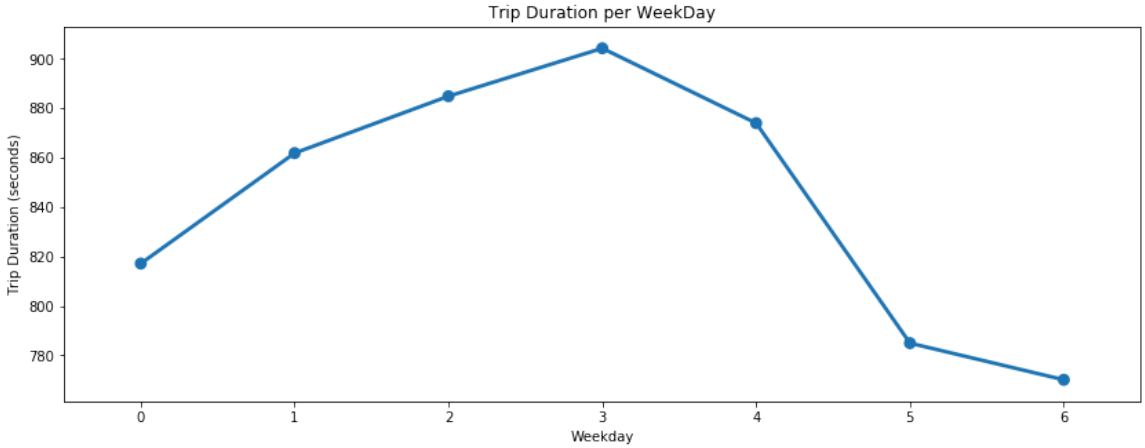


- Average trip duration is lowest at 6 AM when there is minimal traffic on the roads.
- Average trip duration is generally highest around 3 PM during the busy streets.
- Trip duration on an average is similar during early morning hours i.e. before 6 AM & late evening hours i.e. after 6 PM.

2.Trip duration per WeekDay

Let's now analyze the pattern of trip duration during the week.

In [11]:
```
plt.figure(figsize = (14,5))
group2 = df.groupby('weekday_num').trip_duration.mean()
sns.pointplot(group2.index, group2.values)
plt.ylabel('Trip Duration (seconds)')
plt.xlabel('Weekday')
plt.title('Trip Duration per WeekDay')
plt.show()
```



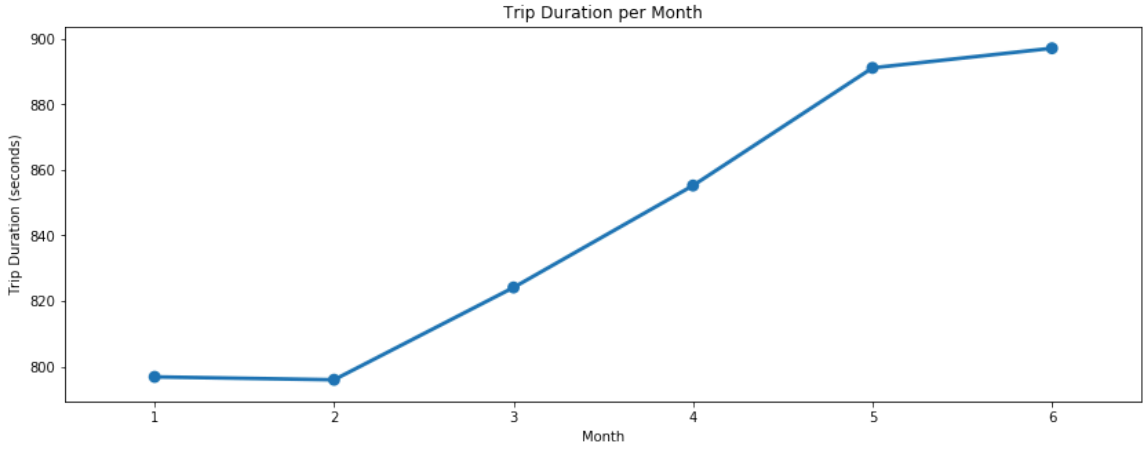
We can see that trip duration is almost equally distributed across the week on a scale of 0-1000 minutes with minimal difference in the duration times. Also, it is observed that trip duration on thursday is longest among all days.

3.Trip duration per Month

Let's take a look at the trip duration pattern with respect to the different months.

In [12]:
```
plt.figure(figsize = (14,5))
group3 = df.groupby('month').trip_duration.mean()
sns.pointplot(group3.index, group3.values)
plt.ylabel('Trip Duration (seconds)')
plt.xlabel('Month')
```

```
plt.title('Trip Duration per Month')
plt.show()
```

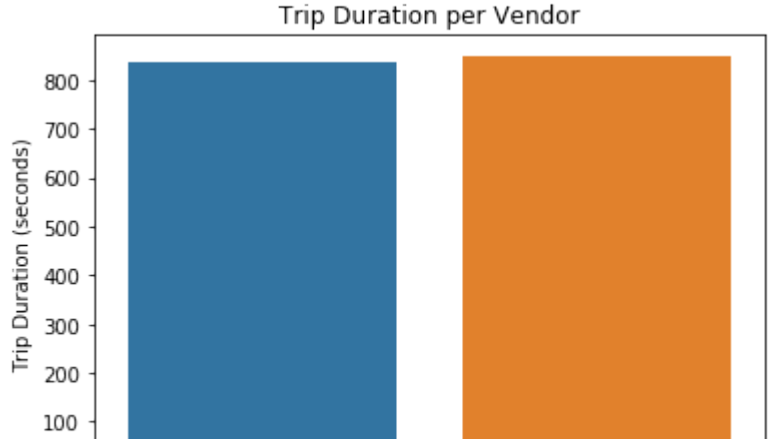


- We can see an increasing trend in the average trip duration along with each subsequent month.
- The duration difference between each month is not much. It has increased gradually over a period of 6 months.
- It is lowest during february when winters starts declining.
- There might be some seasonal parameters like wind/rain which can be a factor of this gradual increase in trip duration over a period. Like May is generally the considered as the wettest month in NYC and which is inline with our visualization. As it generally takes longer on the roads due to traffic jams during rainy season. So natually the trip duration would increase towards April May and June.

4.Trip duration per vendor

We can also look at the average difference between the trip duration for each vendor. However we do know that vendor 2 has larger share of the market. Let's visualize.

In [14]:
```
group4 = df.groupby('vendor_id').trip_duration.mean()
sns.barplot(group4.index, group4.values)
plt.ylabel('Trip Duration (seconds)')
plt.xlabel('Vendor')
plt.title('Trip Duration per Vendor')
plt.show()
```



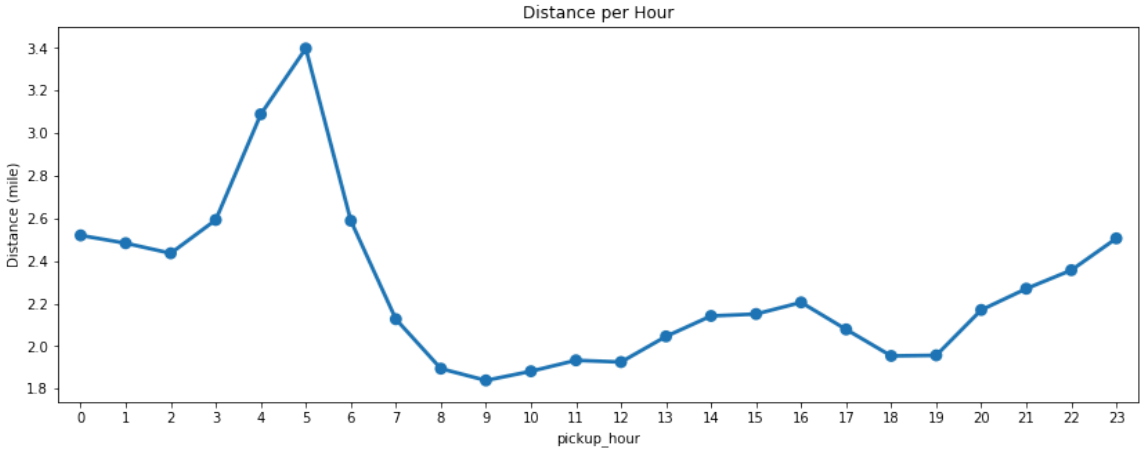
Vendor 2 takes the crown. Average trip duration for vendor 2 is higher than vendor 1 by a quite low margin.

5.Distance per hour

Now, let us check how the distance is distributed against different variables. We know that trip distance must be more or less proportional to the trip duration if we ignore general traffic and other stuff on the road. Let's visualize this for each hour now.

Since we have already done the outlier analysis for this variable as well. We can take the mean as aggregate measure for our visualizations.

In [15]:
```
plt.figure(figsize = (14,5))
group5 = df.groupby('pickup_hour').distance.mean()
sns.pointplot(group5.index, group5.values)
plt.ylabel('Distance (mile)')
plt.title('Distance per Hour')
plt.show()
```



- Trip distance is highest during early morning hours which can account for some things like:

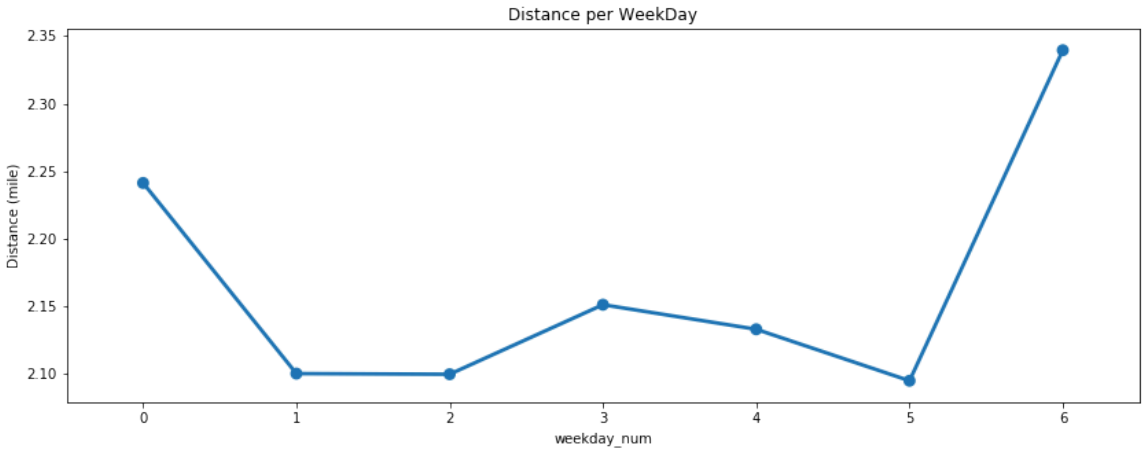
1. Outstation trips taken during the weekends.
2. Longer trips towards the city airport which is located in the outskirts of the city.

- Trip distance is fairly equal from morning till the evening varying around 2 - 2.5 mile.
- It starts increasing gradually towards the late night hours starting from evening till 5 AM and decrease steeply towards morning.

6.Distance per WeekDay

Let's analyze the average trip distance covered on each day of the week.

In [16]:
```
plt.figure(figsize = (14,5))
group6 = df.groupby('weekday_num').distance.mean()
sns.pointplot(group6.index, group6.values)
plt.ylabel('Distance (mile)')
plt.title('Distance per WeekDay')
plt.show()
```

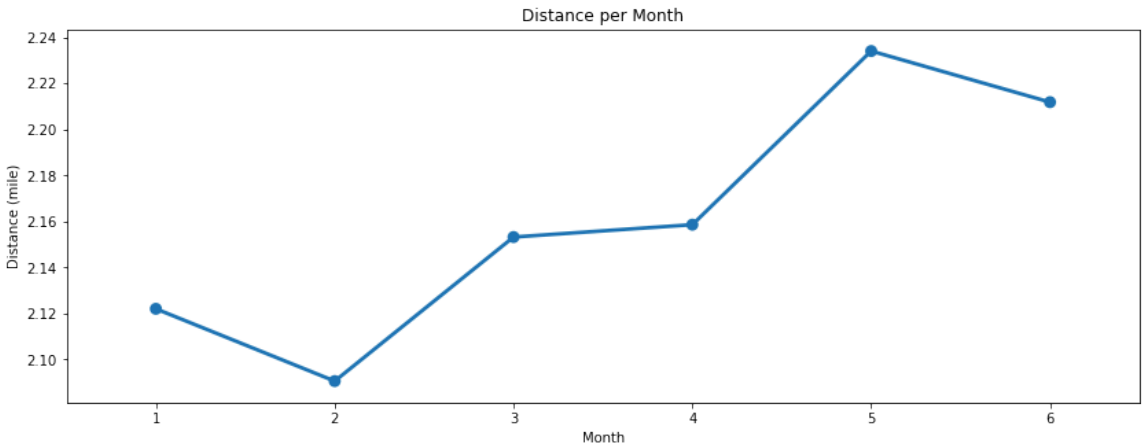


So it's a fairly equal distribution with average distance metric verying around 2 mile/h with Sunday being at the top may be due to outstation trips or night trips towards the airport.

7.Distance per Month

Now we will look at the average trip distance covered per month.

In [18]:
```
plt.figure(figsize = (14,5))
group7 = df.groupby('month').distance.mean()
sns.pointplot(group7.index, group7.values)
plt.ylabel('Distance (mile)')
plt.xlabel('Month')
plt.title('Distance per Month')
plt.show()
```

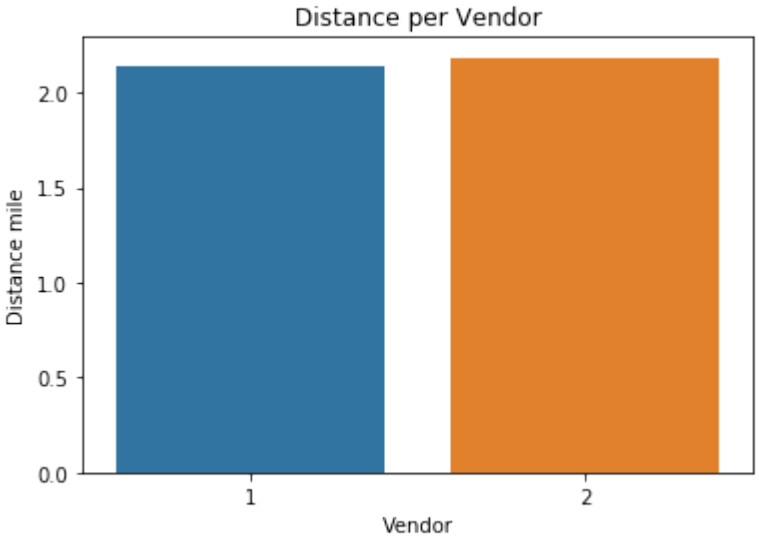


Here also the distibution is almost equivalent, varying mostly around 3.5 km/h with 5th month being the highest in the average distance and 2nd month being the lowest.

8.Distance per Vendor

Let's check how both the vendors have covered the average distance during the trips

In [19]:
```
group8 = df.groupby('vendor_id').distance.mean()
sns.barplot(group8.index, group8.values)
plt.ylabel("Distance mile")
plt.xlabel("Vendor")
plt.title('Distance per Vendor')
plt.show()
```

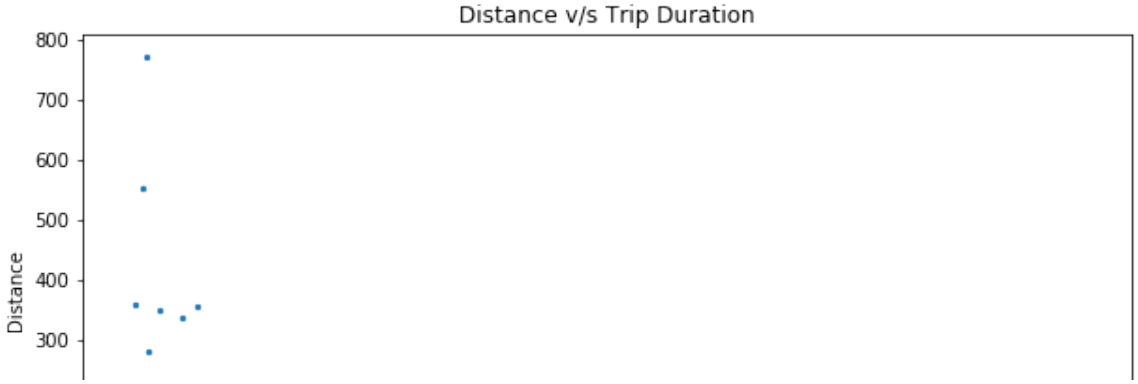


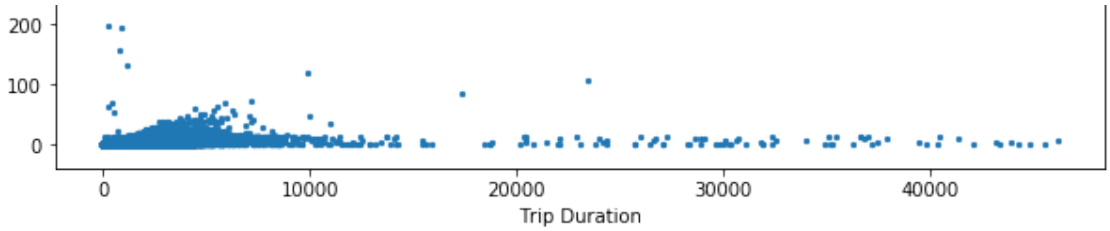
This is more or less same picture with both the vendors. Nothing more to analyze in this.

9.Distance v/s Trip duration

Let's visualize the relationship between Distance covered and respective trip duration.

In [20]:
```
plt.figure(figsize = (10,5))
plt.scatter(df.trip_duration, df.distance , s=5, alpha=1)
plt.ylabel('Distance')
plt.xlabel('Trip Duration')
plt.title('Distance v/s Trip Duration')
plt.show()
```

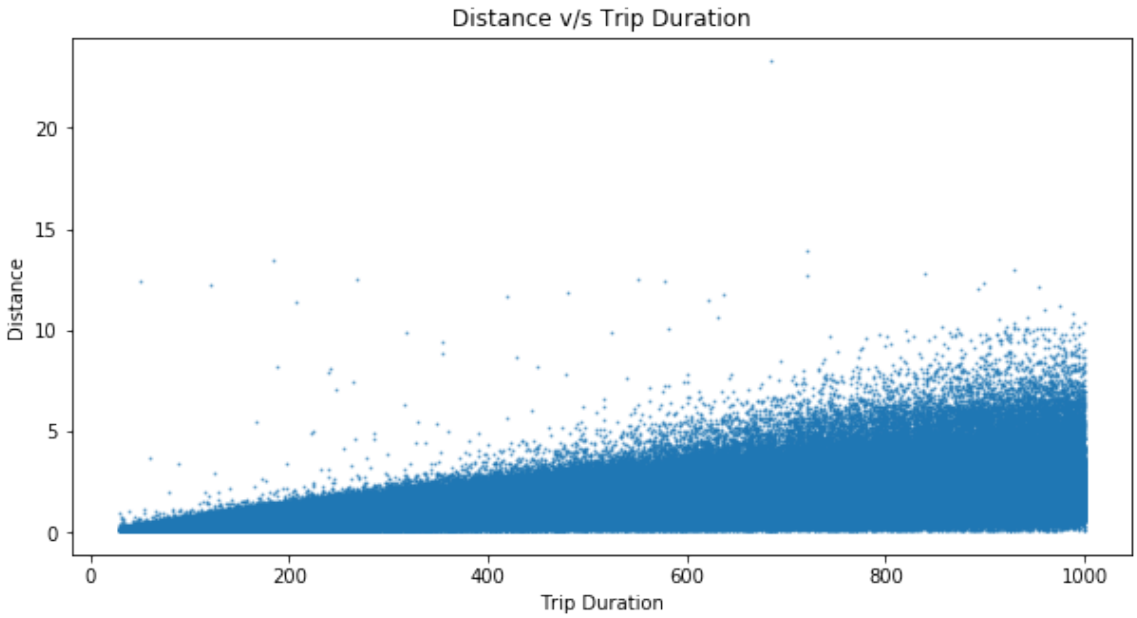




- There are lots of trips which covered negligible distance but clocked more than 20,000 seconds in terms of the Duration.
- Initially there is some proper correlation between the distance covered and the trip duration in the graph. but later on it all seems uncorrelated.
- There were few trips which covered huge distance of approx 120 miles within very less time frame, which is unlikely and should be treated as outliers.

Let's focus on the graph area where distance is < 30 mile and duration is < 1000 seconds.

In [21]:
```
plt.figure(figsize = (10,5))
dur_dist = df.loc[(df.distance < 30) & (df.trip_duration < 1000), ['distance'
plt.scatter(dur_dist.trip_duration, dur_dist.distance , s=1, alpha=0.5)
plt.ylabel('Distance')
plt.xlabel('Trip Duration')
plt.title('Distance v/s Trip Duration')
plt.show()
```



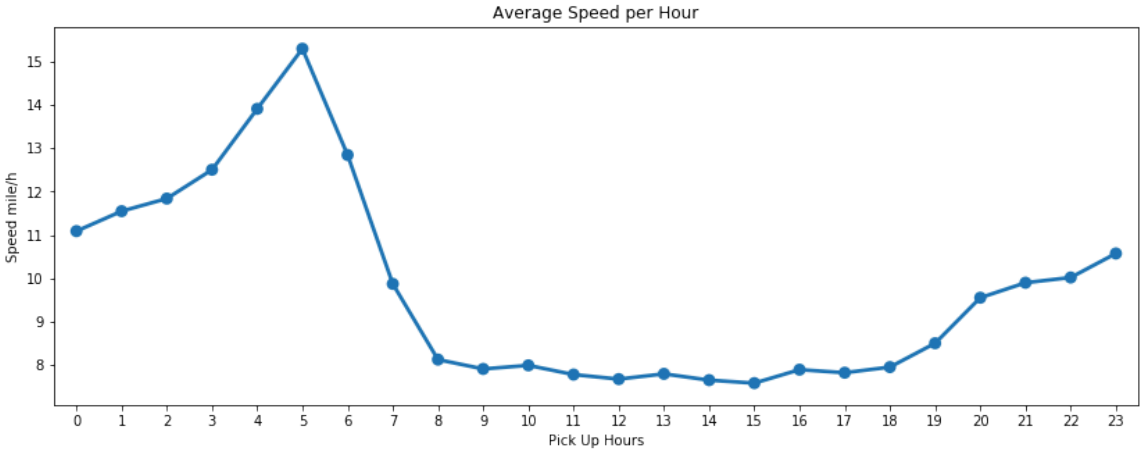
- There should have been a linear relationship between the distance covered and trip duration on an average but we can see dense collection of the trips in the lower right corner which showcase many trips with the inconsistent readings.

We should remove those trips which covered 0 mile distance but clocked more than 1 minute to make our data more consistent for predictive model. Because if the trip was cancelled after booking, than that should not have taken more than a minute time. This is our assumption.

10.Average speed per hour

Let's look at the average speed of NYC Taxi per hour.

In [24]:
```
plt.figure(figsize = (14,5))
group9 = df.groupby('pickup_hour').speed.mean()
sns.pointplot(group9.index, group9.values)
plt.xlabel('Pick Up Hours')
plt.ylabel('Speed mile/h')
plt.title('Average Speed per Hour')
plt.show()
```

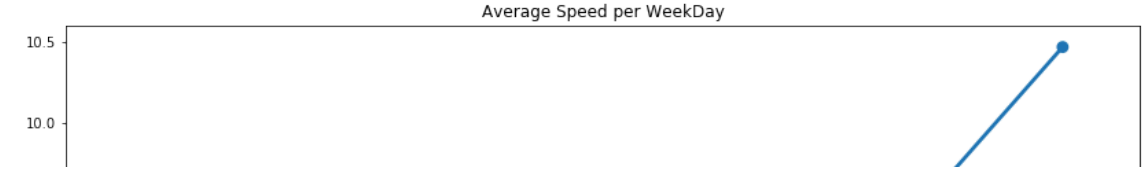


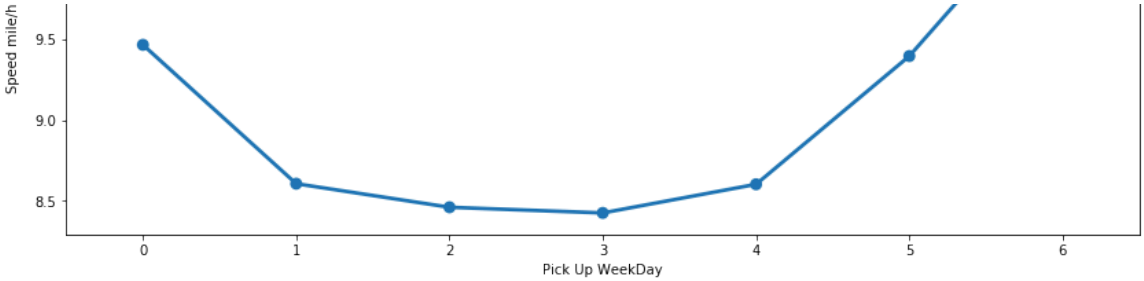
- The average trend is totally inline with the normal circumstances.
- Average speed tend to increase after late evening and continues to increase gradually till the late early morning hours.
- Average taxi speed is highest at 5 AM in the morning, then it declines steeply as the office hours approaches.
- Average taxi speed is more or less same during the office hours i.e. from 8 AM till 6PM in the evening.

11.Average speed per weekday

Let's visualize that on an average what is the speed of a taxi on any given weekday.

In [26]:
```
plt.figure(figsize = (14,5))
group10 = df.groupby('weekday_num').speed.mean()
sns.pointplot(group10.index, group10.values)
plt.xlabel('Pick Up WeekDay')
plt.ylabel('Speed mile/h')
plt.title('Average Speed per WeekDay')
plt.show()
```





- Average taxi speed is higher on weekend as compared to the weekdays which is obvious when there is mostly rush of office goers and business owners.
- Even on monday the average taxi speed is shown higher which is quite surprising when it is one of the most busiest day after the weekend. There can be several possibility for such behaviour

1. Lot of customers who come back from outstation in early hours of Monday before 6 AM to attend office on time.
2. Early morning hours customers who come from the airports after vacation to attend office/business on time for the coming week.

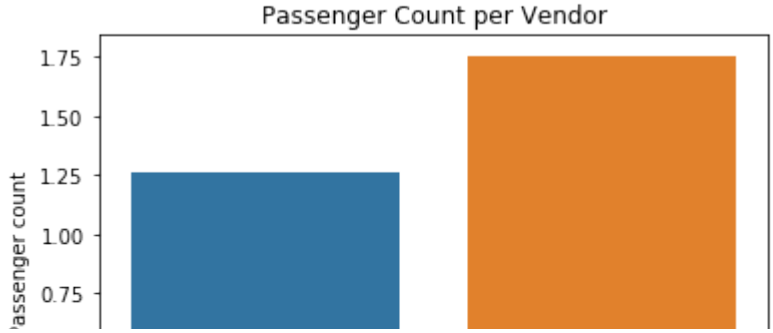
- There could be some more reasons as well which only a local must be aware of.
- We also can't deny the anomalies in the dataset. which is quite cumbersome to spot in such a large dataset.

12.Passenger count per vendor

Let's try some different metric in the series i.e. passenger count. We will plot it agaist the vendor only because it will not be much helpful to plot it against hour, weekday or month like others as the passenger count should be a whole number and not a ratio.

we will take mean as the aggregate measure because we already did the outlier analysis on this metric. So our results woudn't be affected by some extreme values. Also if we take median than it will return only 1 because majorty of the trips have been taken by single passenger. Let's take a look about it's distribution.

In [27]:
```
group9 = df.groupby('vendor_id').passenger_count.mean()
sns.barplot(group9.index, group9.values)
plt.ylabel('Passenger count')
plt.xlabel('Vendor Id')
plt.title('Passenger Count per Vendor')
plt.show()
```



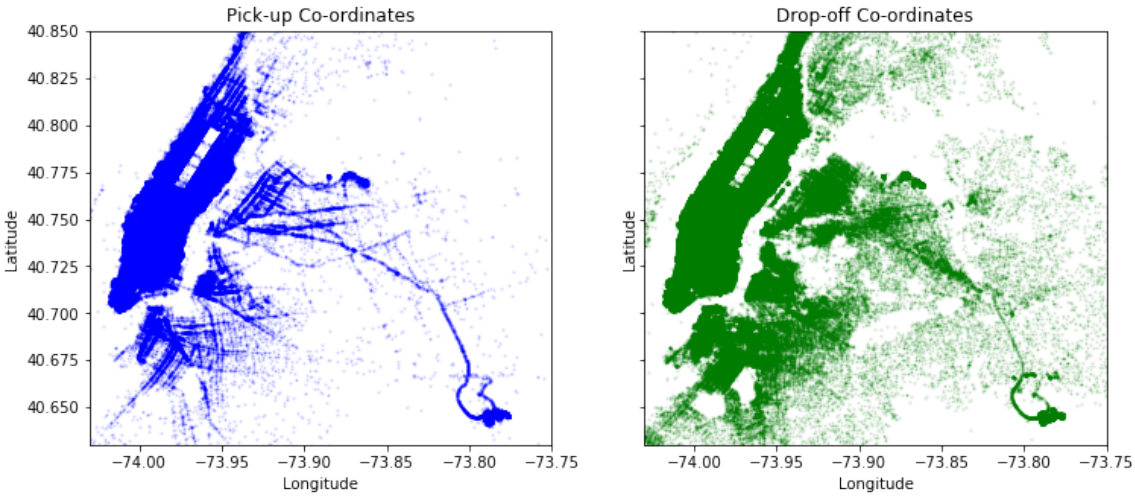
Clear difference between the two operators for the average passenger count in all trips. It seems that vendor 2 trips generally consist of 2 passengers as compared to the vendor 1 with 1 passenger. Let's bifurcate it further.

- It seems that most of the big cars are served by the Vendor 2 including minivans because other than passenger 1, vendor 2 has majority in serving more than 1 passenger count and that explains it greater share of the market.

13.Pick Up Points v/s Dropoff Points

Here we take a look at the spread of Pickup co-ordinates and Drop-off co-ordinates across city

In [6]:
```
city_long_border = (-74.03, -73.75)
city_lat_border = (40.63, 40.85)
fig, ax = plt.subplots(ncols=2, sharex=True, sharey=True,figsize = (12,5))
ax[0].scatter(df['pickup_longitude'].values, df['pickup_latitude'].values,
color='blue', s=1, label='train', alpha=0.1)
ax[1].scatter(df['dropoff_longitude'].values, df['dropoff_latitude'].values,
color='green', s=1, label='train', alpha=0.1)
ax[1].set_title('Drop-off Co-ordinates')
ax[0].set_title('Pick-up Co-ordinates')
ax[0].set_ylabel('Latitude')
ax[0].set_xlabel('Longitude')
ax[1].set_ylabel('Latitude')
ax[1].set_xlabel('Longitude')
plt.ylim(city_lat_border)
plt.xlim(city_long_border)
plt.show()
```



- In the Pickup plot we can see the spread is mostly concentrated on Manhattan

area.

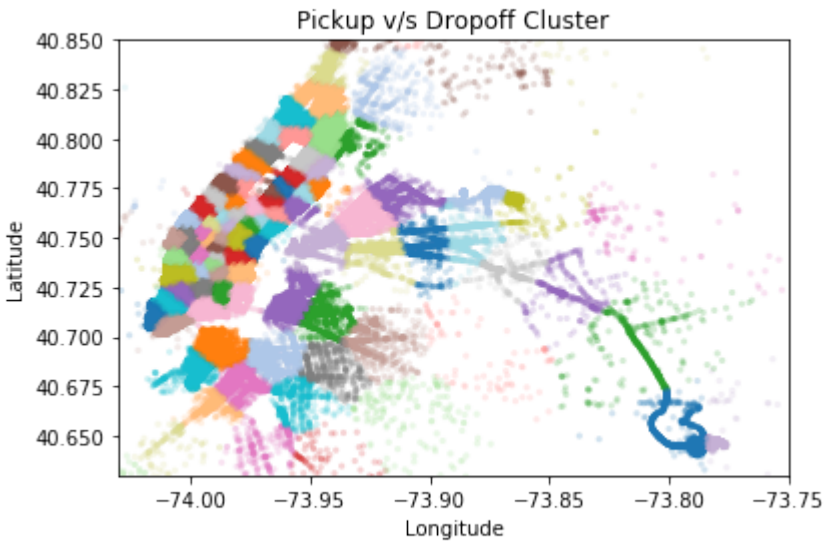
- We can say Manhattan is quite populated for a pickup spot.
- Whereas the drop zone is quite spreaded out compared to pickup.
- As analysed in distance analysis the average distance is 2.1 miles which explain the heavy intensity of drop in Manhattan itself.

Clustering

In [7]:
```
coords = np.vstack((df[['pickup_latitude', 'pickup_longitude']].values,
                    df[['dropoff_latitude', 'dropoff_longitude']].values))
```

In [8]:
```
from sklearn.cluster import MiniBatchKMeans
sample_ind = np.random.permutation(len(coords))[:500000]
kmeans = MiniBatchKMeans(n_clusters=100, batch_size=10000).fit(coords[sample_
df.loc[:, 'pickup_cluster'] = kmeans.predict(df[['pickup_latitude', 'pickup_l
df.loc[:, 'dropoff_cluster'] = kmeans.predict(df[['dropoff_latitude', 'dropof
```

In [9]:
```
fig, ax = plt.subplots(ncols=1, nrows=1)
ax.scatter(df.pickup_longitude.values, df.pickup_latitude.values, s=10, lw=0,
           c=df.pickup_cluster.values, cmap='tab20', alpha=0.2)
ax.set_xlim(city_long_border)
ax.set_ylim(city_lat_border)
ax.set_xlabel('Longitude')
ax.set_ylabel('Latitude')
ax.set_title('Pickup v/s Dropoff Cluster')
plt.show()
```



As we can see, the clustering results in a partition which is somewhat similar to the way NY is divided into different neighborhoods. We can see Upper East and West side of Central park in gray and pink respectively. West midtown in blue, Chelsea and West Village in brown, downtown area in blue, East Village and SoHo in purple.

The airports JFK and La LaGuardia have there own cluster, and so do Queens and Harlem. Brooklyn is divided into 2 clusters, and the Bronx has too few rides to be separated from Harlem.

Feature Engineering

After looking at the dataset from different perspectives. Let's prepare our dataset before training our model. Since our dataset do not contain very large number of dimensions. We will first try to use feature selection instead of the feature extraction technique.

1.Feature Selection

We will use technique to select the best features to train our model.

Here the biggest question is which columns are useful for model train

In [20]:
```
df.dtypes
```

Out[20]:
```
id                   object
vendor_id             int64
pickup_datetime      object
dropoff_datetime     object
passenger_count       int64
pickup_longitude     float64
pickup_latitude      float64
dropoff_longitude    float64
dropoff_latitude     float64
store_and_fwd_flag    object
trip_duration          int64
distance             float64
```

```
speed                float64
weekday               object
month                  int64
weekday_num            int64
pickup_hour            int64
pickup_cluster         int32
dropoff_cluster        int32
dtype: object
```

- id column has unique values which means its no use to take id in model training
- Columns such as pickup_datetime,dropoff_datetime has object dtype and the values are datetype which may not able to evaluate by model which can affect in accuracy.
- Therefore pickup_datetime is been converted into weekday, month, weekday_num, pickup_hour
- pickup_longitude,pickup_latitude and dropoff_longitude,dropoff_latitude are the columns which are dependent on each other.
- This columns does not mean much if pass as individual columns.
- Other than this store_and_fwd_flag is just a service info column.
- Excluding this columns the other columns such as vendor_id, passenger_count, distance, speed correlate with duration which can be used in model train.

In [22]:
```
df.shape
```

Out[22]: (1393219, 19)

In [44]:
```
x = df.iloc[:, [1, 4, 11, 12, 14, 15, 16, 17, 18]].values
y = df.iloc[:,10].values
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,rando
```

In [30]:
```
from scipy.stats import pearsonr
df1 = pd.DataFrame(np.concatenate((x_train,y_train.reshape(len(y_train),1)),a
df1.columns = df1.columns.astype(str)

features = df1.iloc[:,:9].columns.tolist()
target = df1.iloc[:,9].name

correlations = {}
for f in features:
    data_temp = df1[[f,target]]
    x1 = data_temp[f].values
    x2 = data_temp[target].values
    key = f + ' vs ' + target
    correlations[key] = pearsonr(x1,x2)[0]

data_correlations = pd.DataFrame(correlations, index=['Value']).T
data_correlations.loc[data_correlations['Value'].abs().sort_values(ascending=
```

Out[30]:

	Value
2 vs 9	0.725760
4 vs 9	0.056251

7 vs 9 -0.051166

3 vs 9 0.047136

5 vs 9 -0.032208

6 vs 9 0.027374

8 vs 9 -0.021245

1 vs 9 0.017286

0 vs 9 0.009772

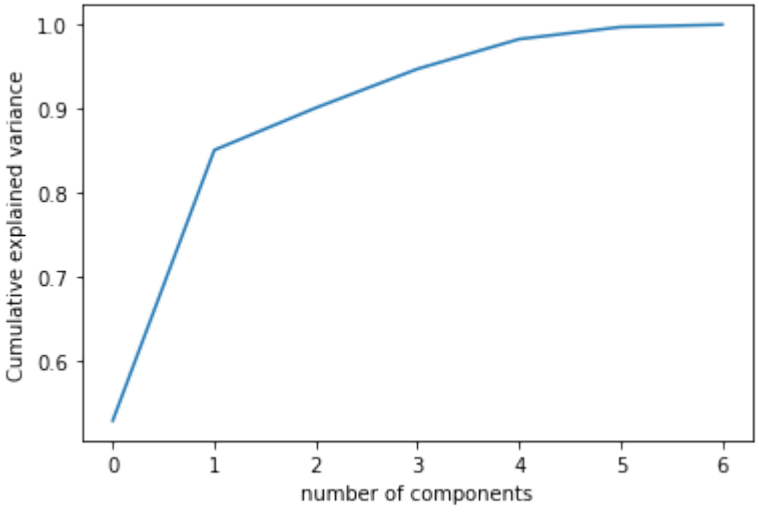
Column 7 and 8 shows negative correlation which means its not useful in model training. It can leave reverse impact on model

2.Feature Extraction

Split Data

In [4]:
```
x = df.iloc[:, [1, 4, 11, 12, 14, 15, 16]].values
y = df.iloc[:,10].values
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,rando
```

In [32]:
```
from sklearn.decomposition import PCA
pca = PCA().fit(x_train)
plt.plot(np.cumsum(pca.explained_variance_ratio_))
plt.xlabel("number of components")
plt.ylabel("Cumulative explained variance")
plt.show()
```



In [34]:
```
arr = np.cumsum(np.round(pca.explained_variance_ratio_, decimals=4)*100)
list(zip(range(1,len(arr)), arr))
```

Out[34]:
```
[(1, 52.81),
 (2, 85.03999999999999),
 (3, 90.08),
 (4, 94.71)
```

```
(4, 94.71),
(5, 98.25999999999999),
(6, 99.69999999999999)]
```

Here we can see that 6 variables are sufficient for capturing atleast 99% of the variance in the training dataset. Hence we will use the same set of variables(i.e this model does not require the use of PCA).

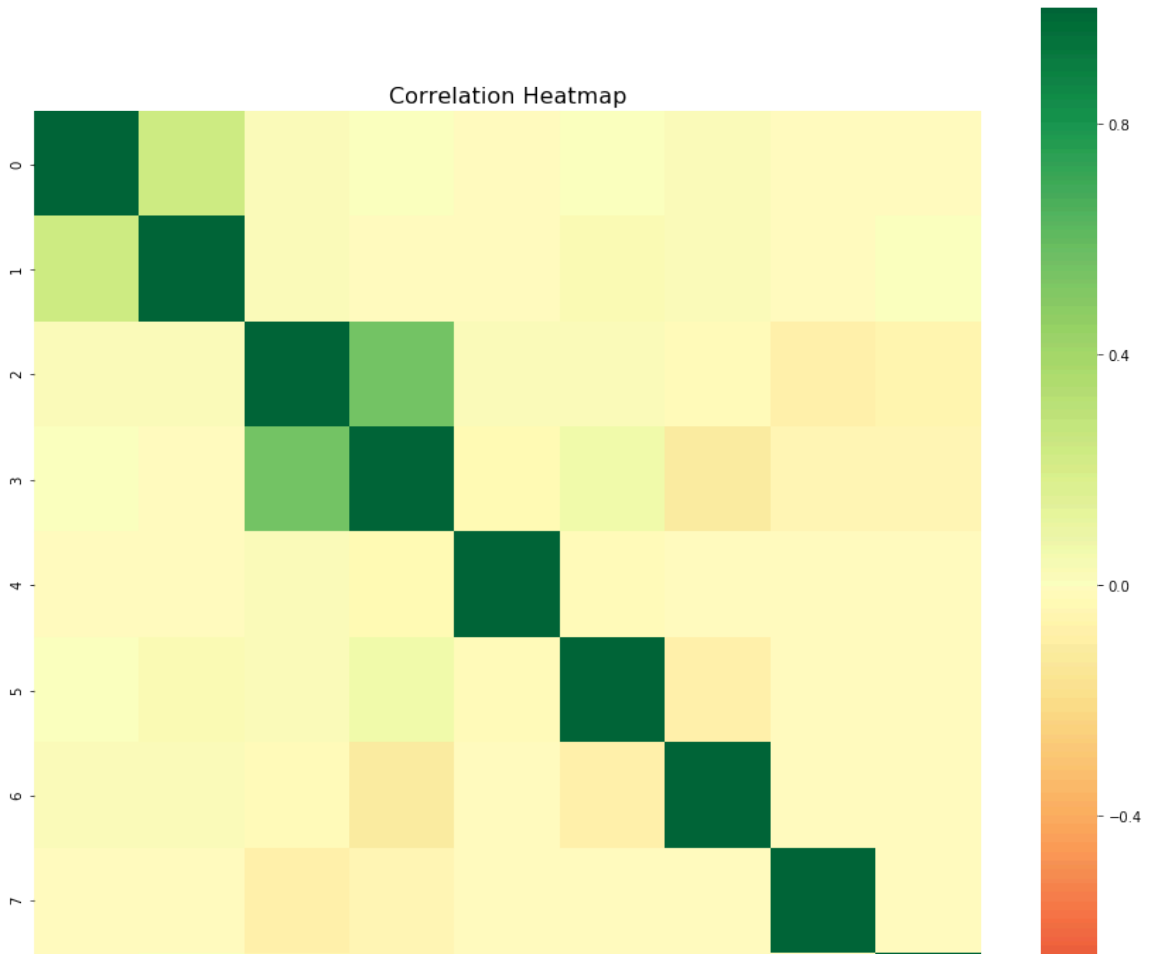
3.Correlation Analysis

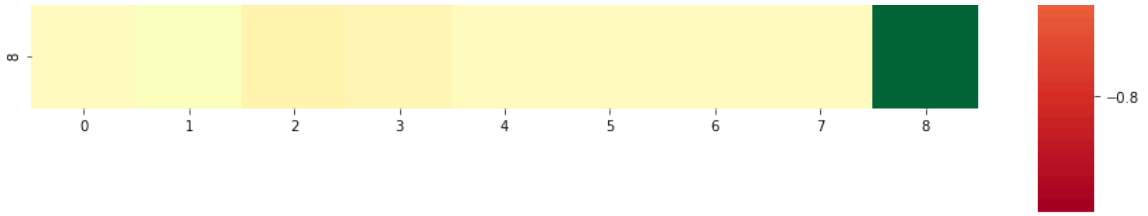
Correlation analysis is a method of statistical evaluation used to study the strength of a relationship between two or more, numerically measured, continuous variables. This analysis is useful when we need to check if there are possible connections between variables. We will utilize Heatmap for our analysis.

Heatmap

A heatmap is a graphical representation of data that uses a system of color-coding to represent statistical relationship between different values.

In [45]:
```
plt.figure(figsize=(15,15))
corr = pd.DataFrame(x_train[:,0:]).corr()
corr.index = pd.DataFrame(x_train[:,0:]).columns
sns.heatmap(corr, cmap='RdYlGn', vmin=-1, vmax=1, square=True)
plt.title("Correlation Heatmap", fontsize=16)
plt.show()
```





- Some combinations of features shows slight correlation.
- But most of the features shows no correlation
- There is no negative correlation

Model

We need a model to train on our dataset to serve our purpose of prediciting the NYC taxi trip duration given the other features as training and test set. Since our dependent variable contains continous values so we will use regression technique to predict our output. We will try almost every Regression model (except SVR)

1.Data Cleaning

- As we analysed there is no null value but there are many outliers in Speed, Distance, and Trip Duration.
- This Outliers need to be removed, if not cleaned it can fluctuate the model.

The Processes of Cleaning Data is done in R language.

R Code

2.Model Training

1.Multiple Linear Regression

It is used to explain the relationship between one continuous dependent variable and two or more independent variables.

In [5]:
```
start_time = time.time()
lm_regression = LinearRegression()
lm_regression = lm_regression.fit(x_train, y_train)
end_time = time.time()
lm_time = (end_time - start_time)
print(f"Time taken to train linear regression model : {lm_time} seconds")
```

```
Time taken to train linear regression model : 0.8252334594726562 seconds
```

In [6]:

```
trips = lm_regression.predict(x_test)
```

In [7]:
```
predictions = pd.DataFrame({'Actual': y_test.flatten(), 'Predicted': trips.fl
```

In [8]:
```
predictions
```

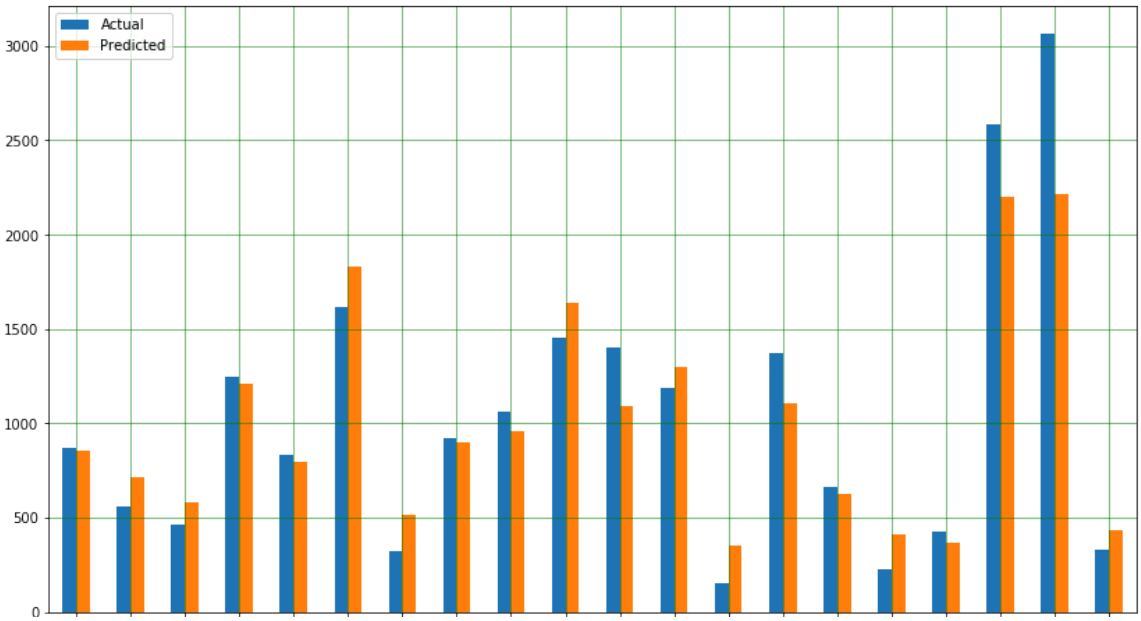
Out[8]:

	Actual	Predicted
0	309	26.635361
1	552	771.837903
2	970	874.028286
3	1255	1333.048618
4	428	657.809462
...	...	...
278639	915	863.914613
278640	1047	1171.771246
278641	707	770.071438
278642	219	-505.426834
278643	265	301.397268

278644 rows × 2 columns

Accuracy Metrics

In [11]:
```
predictions.sample(20).plot(kind='bar',figsize=(14,8))
plt.grid(which='major', linestyle='-', linewidth='0.5', color='green')
plt.grid(which='minor', linestyle=':', linewidth='0.5', color='black')
plt.show()
```



5343 145793 171840 228967 168215 20645 36585 210032 251260 172116 256490 25103 241547 165550 247822 79570 24908 89146 206062 68099

- From the above graph, we can see that there is quite difference between predicted values and Actual values

Lets check the r2_score value

In [12]:
```
lm_score = r2_score(y_test, trips)
print(lm_score)
```

```
0.6843777737519032
```

- The r2_score is above 0.5 which is a good thing but it is quite low for a regression model.

Let's Move towards next Model

2.Decision Tree

Decision tree is one of the predictive modeling approaches used in statistics, data mining and machine learning. It uses a decision tree (as a predictive model) to go from observations about an item (represented in the branches) to conclusions about the item's target value (represented in the leaves)

In [13]:
```
start_time = time.time()
dt_regression = DecisionTreeRegressor()
dt_regression = dt_regression.fit(x_train, y_train)
end_time = time.time()
dt_time = (end_time - start_time)
print(f"Time taken to train Decision tree model : {dt_time} seconds")
```

```
Time taken to train Decision tree model : 22.30292558670044 seconds
```

In [14]:
```
trips = dt_regression.predict(x_test)
```

In [15]:
```
predictions = pd.DataFrame({'Actual': y_test.flatten(), 'Predicted': trips.fl
```

In [16]:
```
predictions
```

Out[16]:

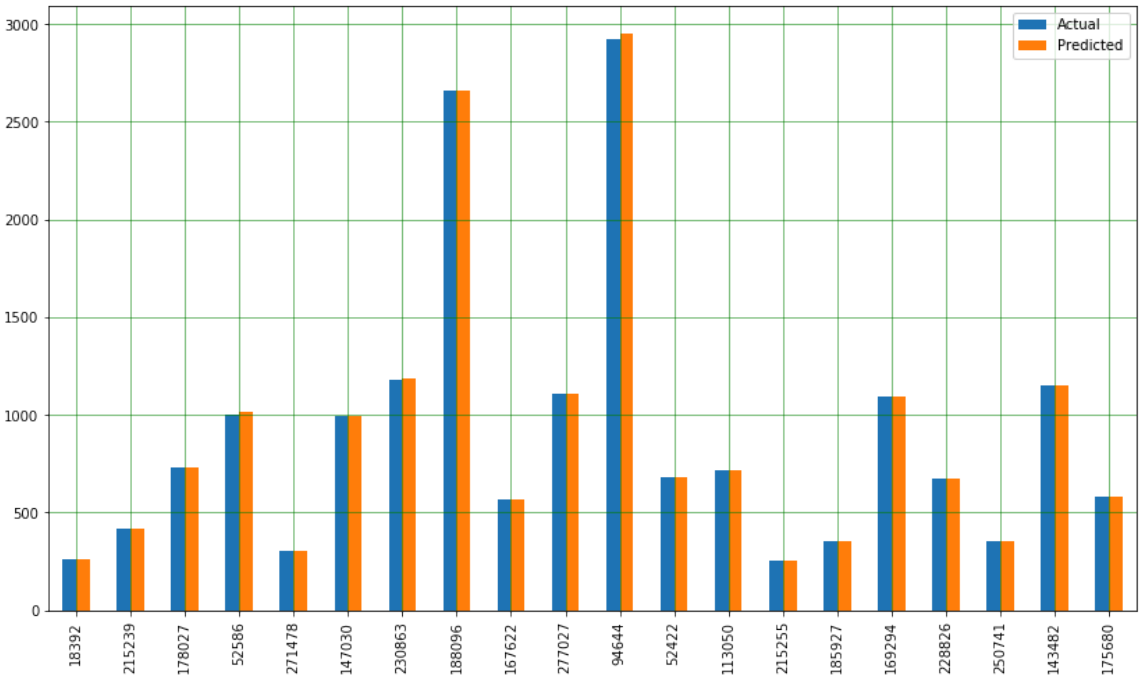
	Actual	Predicted
0	309	309.0
1	552	551.0
2	970	972.0
3	1255	1259.0
4	428	428.0

...	...	...
278639	915	916.0
278640	1047	1028.0
278641	707	710.0
278642	219	219.0
278643	265	265.0

278644 rows × 2 columns

Accuracy Metrics

In [17]:
```
predictions.sample(20).plot(kind='bar',figsize=(14,8))
plt.grid(which='major', linestyle='-', linewidth='0.5', color='green')
plt.grid(which='minor', linestyle=':', linewidth='0.5', color='black')
plt.show()
```



- From the above graph, we can see that difference is quite low between predicted values and Actual values

Lets check the r2_score value

In [18]:
```
dt_score = r2_score(y_test, trips)
print(dt_score)
```

```
0.9939377561956979
```

- The r2 score is best but the time taken for execution is high

So, Let's check if we can get same accuracy with less execution time in next model.

3.Random Forest

Random forests or random decision forests are an ensemble learning method for classification, regression and other tasks that operates by constructing a multitude of decision trees at training time and outputting the class that is the mode of the classes (classification) or mean prediction (regression) of the individual trees.

In [19]:
```
start_time = time.time()
rf_regression = RandomForestRegressor()
rf_regression = rf_regression.fit(x_train, y_train)
end_time = time.time()
rf_time = (end_time - start_time)
print(f"Time taken to train Random Forest model : {rf_time} seconds")
```

```
Time taken to train Random Forest model : 1542.148060798645 seconds
```

In [20]:
```
trips = rf_regression.predict(x_test)
```

In [21]:
```
predictions = pd.DataFrame({'Actual': y_test.flatten(), 'Predicted': trips.fl
```

In [22]:
```
predictions
```

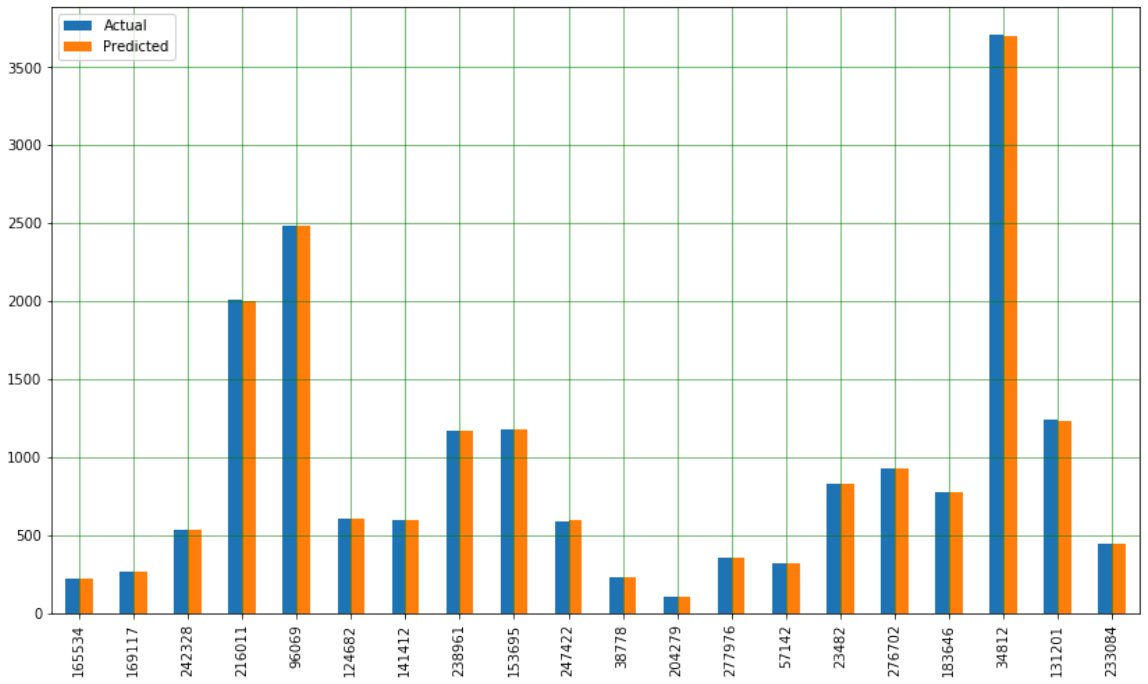
Out[22]:

	Actual	Predicted
0	309	309.23
1	552	551.56
2	970	969.04
3	1255	1255.03
4	428	428.08
...	...	...
278639	915	914.97
278640	1047	1044.90
278641	707	707.00
278642	219	219.77
278643	265	264.93

278644 rows × 2 columns

Accuracy Metrics

In [23]:
```
predictions.sample(20).plot(kind='bar',figsize=(14,8))
plt.grid(which='major', linestyle='-', linewidth='0.5', color='green')
plt.grid(which='minor', linestyle=':', linewidth='0.5', color='black')
plt.show()
```



In [24]:
```
rf_score = r2_score(y_test, trips)
print(rf_score)
```

```
0.996693859823783
```

4.AdaBoost

Adaboost can be used in conjunction with many other types of learning algorithms to improve performance.

In [25]:
```
start_time = time.time()
regression = AdaBoostRegressor()
regression = regression.fit(x_train, y_train)
end_time = time.time()
ad_time = (end_time - start_time)
print(f"Time taken to train AdaBoost model : {ad_time} seconds")
```

```
Time taken to train AdaBoost model : 230.67372965812683 seconds
```

In [26]:
```
trips = regression.predict(x_test)
```

In [27]:
```
predictions = pd.DataFrame({'Actual': y_test.flatten(), 'Predicted': trips.fl
```

In [28]:
```
predictions
```

Out[28]:

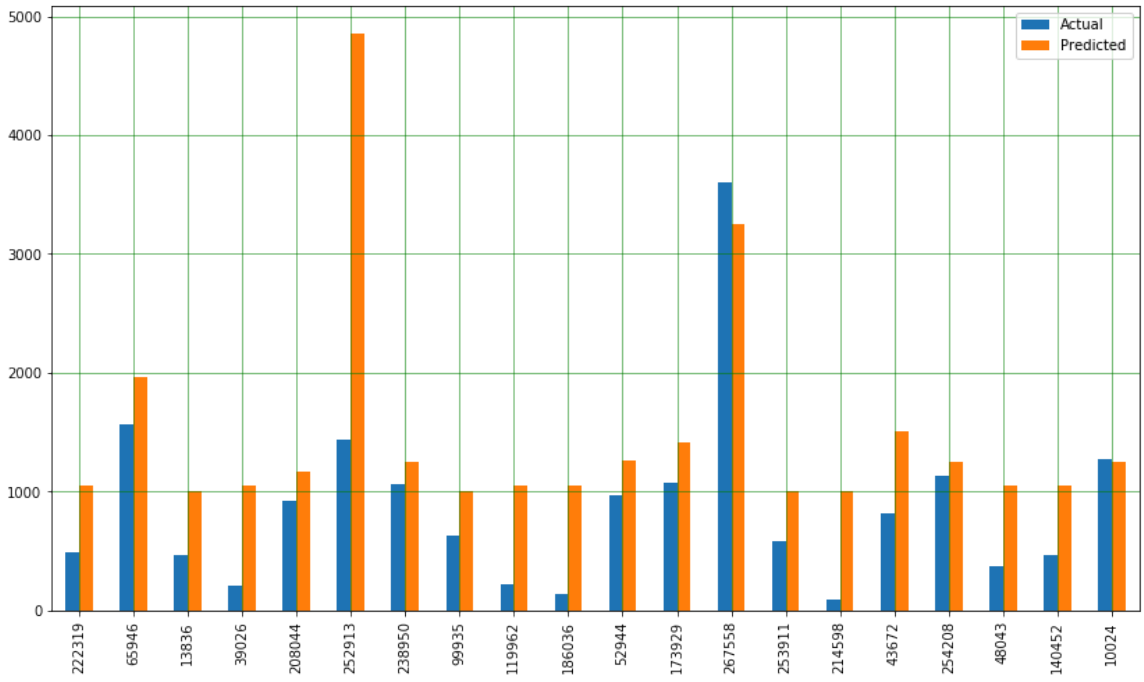
	Actual	Predicted
0	309	1045.751350
1	552	1045.751350
2	970	1045.751350

3	1255	2134.669559
4	428	1001.039409
...	...	...
278639	915	1264.631356
278640	1047	2134.669559
278641	707	1001.039409
278642	219	1001.039409
278643	265	1045.751350

278644 rows × 2 columns

Accuracy Metrics

In [29]:
```
predictions.sample(20).plot(kind='bar',figsize=(14,8))
plt.grid(which='major', linestyle='-', linewidth='0.5', color='green')
plt.grid(which='minor', linestyle=':', linewidth='0.5', color='black')
plt.show()
```



In [30]:
```
ad_score = r2_score(y_test, trips)
print(ad_score)
```

```
0.06616100539656078
```

5.Gradient Boost

Gradient boosting is a machine learning technique for regression and classification problems, which produces a prediction model in the form of an ensemble of weak prediction models, typically decision trees

In [31]:
```
start_time = time.time()
regression = GradientBoostingRegressor()
regression = regression.fit(x_train, y_train)
end_time = time.time()
gd_time = (end_time - start_time)
print(f"Time taken to train Gradient Boost model : {gd_time} seconds")
```

```
Time taken to train Gradient Boost model : 538.529611825943 seconds
```

In [32]:
```
trips = regression.predict(x_test)
```

In [33]:
```
predictions = pd.DataFrame({'Actual': y_test.flatten(), 'Predicted': trips.fl
```

In [34]:
```
predictions
```

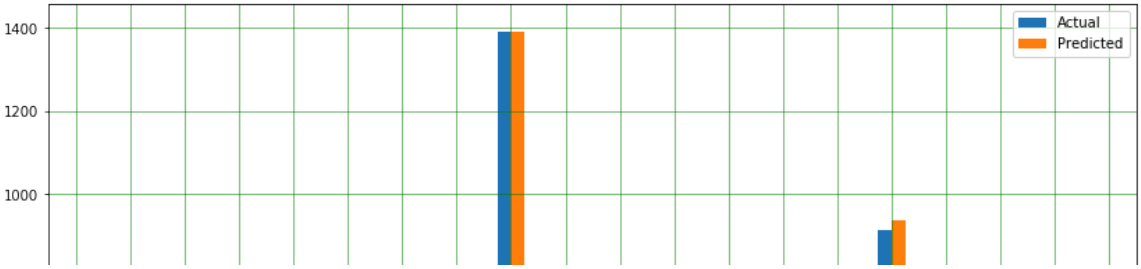
Out[34]:

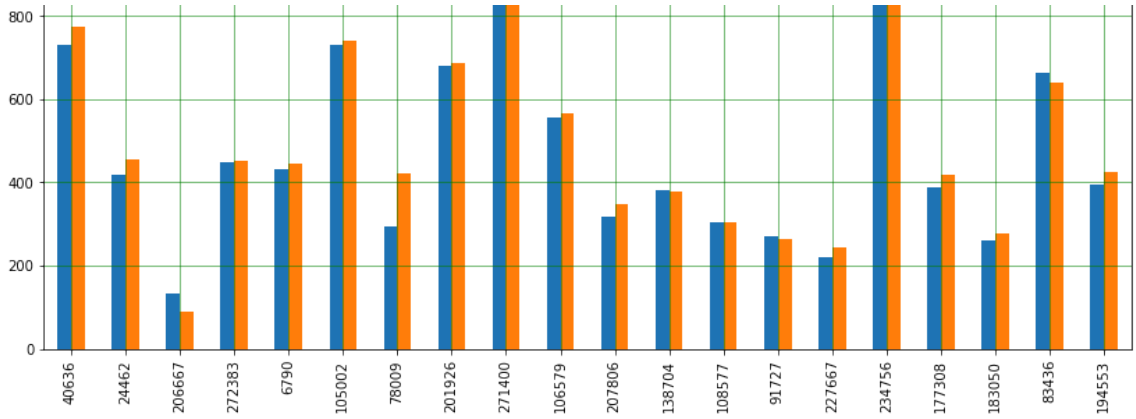
	Actual	Predicted
0	309	301.086941
1	552	596.649072
2	970	991.285772
3	1255	1246.523027
4	428	451.321849
...	...	...
278639	915	951.253493
278640	1047	1056.376899
278641	707	722.786186
278642	219	212.025720
278643	265	270.264769

278644 rows × 2 columns

Accuracy Metrics

In [82]:
```
predictions.sample(20).plot(kind='bar',figsize=(14,8))
plt.grid(which='major', linestyle='-', linewidth='0.5', color='green')
plt.grid(which='minor', linestyle=':', linewidth='0.5', color='black')
plt.show()
```





In [36]:
```
gd_score = r2_score(y_test, trips)
print(gd_score)
```

```
0.9911866103511286
```

6.XGBoost

XGBoost is a decision-tree-based ensemble Machine Learning algorithm that uses a gradient boosting framework.

In [37]:
```
start_time = time.time()
regression = XGBRegressor(objective ='reg:squarederror')
regression = regression.fit(x_train, y_train)
end_time = time.time()
xgb_time = (end_time - start_time)
print(f"Time taken to train XGBoost  model : {xgb_time} seconds")
```

```
Time taken to train XGBoost  model : 211.57350993156433 seconds
```

In [38]:
```
trips = regression.predict(x_test)
```

In [39]:
```
predictions = pd.DataFrame({'Actual': y_test.flatten(), 'Predicted': trips.fl
```

In [40]:
```
predictions
```

Out[40]:

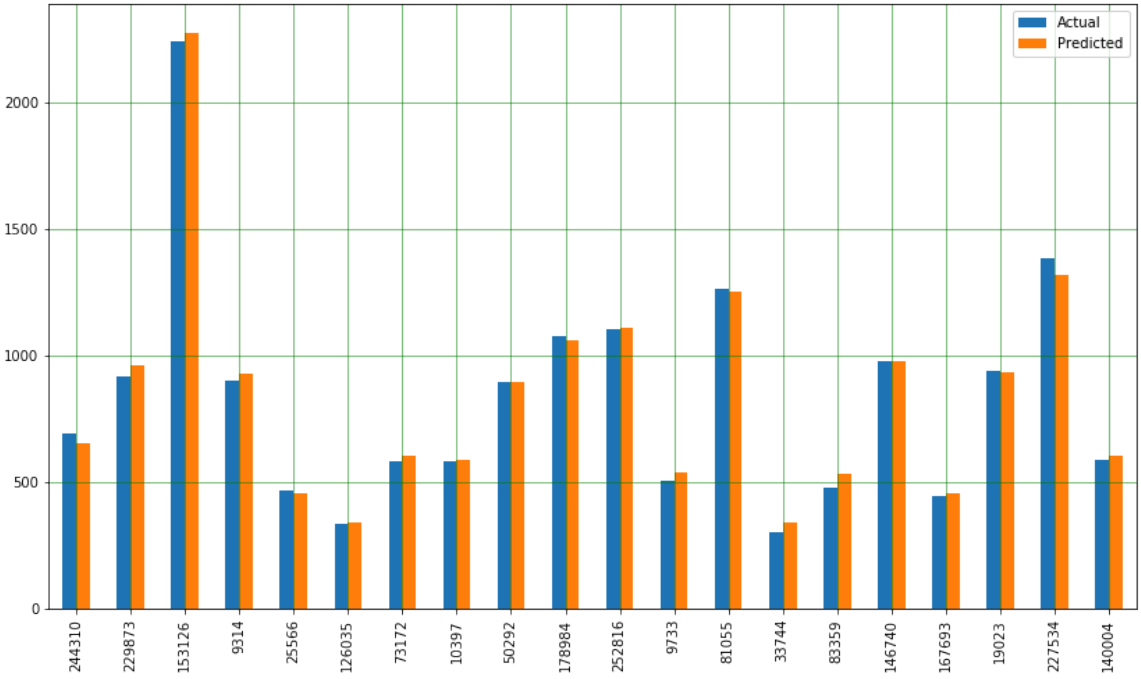
	Actual	Predicted
0	309	328.435364
1	552	566.567932
2	970	974.474976
3	1255	1308.007324
4	428	444.079224
...	...	...
278639	915	953.077637
278640	1047	1114.711060

278641	707	716.730896
278642	219	225.195160
278643	265	252.899933

278644 rows × 2 columns

Accuracy Metrics

In [41]:
```
predictions.sample(20).plot(kind='bar',figsize=(14,8))
plt.grid(which='major', linestyle='-', linewidth='0.5', color='green')
plt.grid(which='minor', linestyle=':', linewidth='0.5', color='black')
plt.show()
```



In [42]:
```
xgb_score = r2_score(y_test, trips)
print(xgb_score)
```

```
0.9913457314056381
```

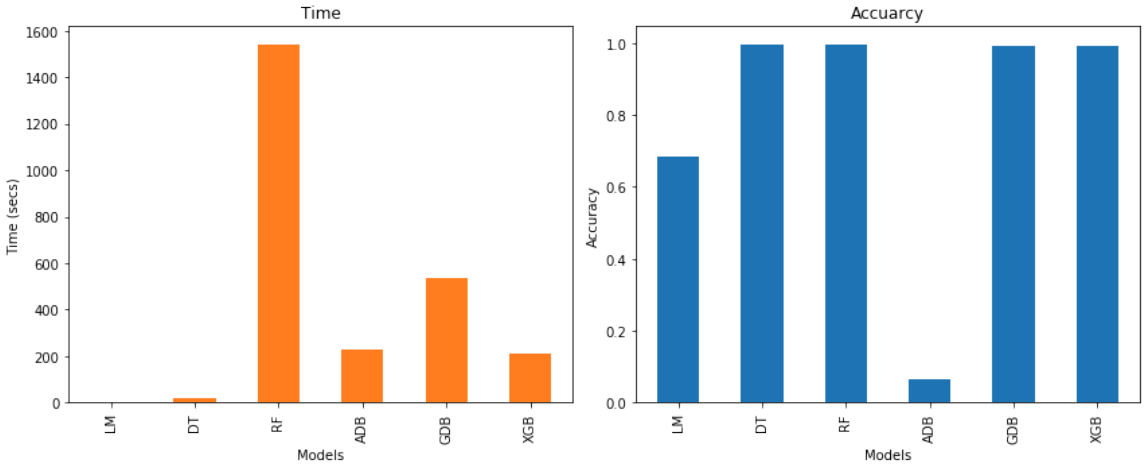
Model Comparison

Lets Analysis all the model we train based on their r2_score and time taken for execution.

In [71]:
```
r2 = [lm_score, dt_score, rf_score, ad_score, gd_score, xgb_score]
tm = [lm_time, dt_time, rf_time, ad_time, gd_time, xgb_time]
comp = pd.DataFrame({'Time': tm, 'Accu': r2})
```

In [81]:
```
label = ['LM', 'DT', 'RF', 'ADB', 'GDB', 'XGB']
fig, axes = plt.subplots(nrows=1, ncols=2,figsize=(12,5))
ax = comp['Time'].plot(kind='bar',title="Time",ax=axes[0],color = (1, 0.5, 0.
ax1 = comp['Accu'].plot(kind='bar',title="Accuarcy",ax=axes[1])
```

```
ax.set_ylabel('Time (secs)')
ax.set_xlabel('Models')
ax.set_xticklabels(label)
ax1.set_ylabel("Accuracy")
ax1.set_xlabel('Models')
ax1.set_xticklabels(label)
fig.tight_layout()
```



From the above fig we can finally decide which Model is best suitable for this dataset.

- We should straight away reject Random Forest as it takes the most amount of time.
- Ada Boost takes less time compared to Random Forest but gives the worst Accuracy from all other.
- Linear Regression takes Least amount of time but doesn't give much accuracy.
- Which leaves with Decision Tree, Gradient Boost and XGBoost.
- All of three have almost same Accuracy but Decision Tree takes least amount of time.

Conclusion

According to the whole data analysis and visualization we have tried six of the best algorithms known and we have came to the conclusion that Decision tree is the best suitable algorithm in this scenario as it gives best accuracy in least amount of time.

So, Here we conclude the analysis with the conclusion that **Decision Tree** algorithm is best suitable algorithm for this dataset.